



A loss matrix-based alternating optimization method for sparse PU learning

Jianfeng Qiu^{a,b,c,d}, Xiaoqiang Cai^{a,b,c,d}, Lei Zhang^{a,b,c,d}, Fan Cheng^{a,b,c,d,*}

^a Information Materials and Intelligent Sensing Laboratory of Anhui Province, Anhui University, Hefei 230039, China

^b School of artificial intelligence, Anhui University, Hefei 230039, China

^c School of Computer Science and Technology, Anhui University, Hefei 230039, Anhui, China

^d Key Laboratory of Intelligent Computing and Signal Processing of Ministry of Education, Anhui University, Hefei 230039, China

ARTICLE INFO

Keywords:

Sparse PU learning
Alternating optimization
Loss matrix
Multi-objective evolutionary algorithm

ABSTRACT

Positive and unlabeled (PU) learning is an important research topic in machine learning area, whose aim is to learn a good classifier from PU data. Due to its wide applications, a variety of PU algorithms with promising performance have been proposed. However, few of them consider PU learning from high-dimensional PU data. To fill the gap, in this work, we focus on designing the sparse PU classifier. However, it is difficult to achieve it, since the labels of U samples are uncertain. To this end, a loss matrix-based alternating optimization method, named LMAO-PU is proposed, where a two-stage alternating optimization idea is suggested to solve the difficulty in sparse PU learning. Firstly, a loss matrix is designed to measure the classification performance of the PU classifier on U samples. Then, a two-stage alternating optimization under the guide of loss matrix is developed. Specifically, the first stage is to optimize the PU model, aiming to address the challenge of labels uncertainty of U samples, where two objectives, *TPR* (True Positive Rate) and a suggested *LRU* (Loss Rate on Unlabeled), are optimized simultaneously. The second stage is to perform the sparse model optimization, where *Sparsity* and *ErrorRate* are adopted as the objectives to obtain the sparse model. The two-stage optimization procedure mentioned above is carried alternately and the sparse PU classifier with high quality is finally achieved. Experiments on 10 high-dimensional datasets demonstrate the superiority of proposed method over six state-of-the-art baselines in terms of sparsity, accuracy and area under ROC curve.

1. Introduction

Positive and unlabeled (PU) learning is an important task in the area of machine learning, whose aim is to train a binary classifier (PU classifier) from only the PU data for predicting the labels of unseen samples. PU learning has applicability in the fields of not only recommender systems [1], but also fake news detection [2] and drug-drug interaction prediction [3], etc. Different from the positive and negative (PN) learning [4], the training data in PU learning only includes the P and U samples, which brings the difficulties for the existing supervised classification algorithms. During the last two decades, many efforts have been devoted to developing a variety of PU algorithms for training efficient PU classifiers, which can be roughly classified into three categories, that is, the sample selection-based approaches, biased-based learning approaches and class prior-based approaches [5]. The sample selection-based methods firstly select a part of samples from U samples as “reliable” negative samples, then to train a classifier on the P and the selected negative samples [6–9]. Different from the first category methods that only considering a part of U samples, the biased-based learning methods are characterized by regarding all U samples

as negative ones with label noise, and then to train a PU classifier on the P and the noisy negative samples [10–13]. As for the class prior-based approaches, each U sample is regarded as the positive or negative sample and the PU learning is converted into a cost-sensitive learning problem to train PU classifiers [14–16]. Recently, with the advantage of multi-objective evolutionary algorithms (MOEAs), Qiu et al. proposed an MOEA-based PU algorithm [17] and the experimental results have demonstrated the competitive performance compared with the state-of-the-art PU algorithms.

Although the existing PU algorithms have shown the promising performance, they may not perform well on complicated classification tasks, such as drug-drug interactions classification [18], disease diagnosis [19] and anomaly detection [20], etc. The reason is that most of them develop PU models by designing different PU algorithms while few of them consider high-dimensional PU learning. In fact, in the scenario of high-dimensional PU data, the curse of dimension brings the new challenge for the existing PU algorithms in training PU classifiers with high quality. To this end, in this paper, we propose a loss matrix-based alternating optimization method for sparse PU learning, in which

* Corresponding author.

E-mail addresses: qiu Jianfeng@ahu.edu.cn (J. Qiu), saiqiang413@gmail.com (X. Cai), zl@ahu.edu.cn (L. Zhang), chengfan@mail.ustc.edu.cn (F. Cheng).

<https://doi.org/10.1016/j.swevo.2022.101174>

Received 16 December 2021; Received in revised form 21 August 2022; Accepted 30 August 2022

Available online 3 September 2022

2210-6502/© 2022 Elsevier B.V. All rights reserved.

two models, PU model and sparse model, are constructed, respectively and optimized alternately for sparse PU learning. Experimental results have demonstrated the superiority of proposed method in tackling high-dimensional PU data situation, compared with the state-of-the-arts. Specifically, the main contributions of this paper is summarized as follows:

- (1) Different from the existing PU algorithms that few of them focus on the PU learning with high-dimensional data, this paper tackles the issue by concentrating on the sparse PU learning. To overcome the challenge of the labels uncertainty of U samples in sparse PU learning, an alternating optimization idea is suggested, where the PU model and the sparse model are alternately optimized to achieve the sparse PU model with high quality.
- (2) Based on the suggested idea, a loss matrix (*LM*) based MOEA, termed LMAO-PU is proposed, where a loss matrix is designed to guide the alternating optimization procedure. In *LM*, each $LM[i, j]$ denotes the classification performance of the i th PU classifier on the j th U sample. Then, with the designed *LM*, a two-stage alternating optimization is implemented in detail. Specifically, the objectives of the first stage are *TPR* and a new designed metric *LRU*, whose aim is to obtain the optimal PU model. While the objectives of the second stage are *Sparsity* and *ErrorRate*, its goal is to achieve the good sparse PU classifier.
- (3) We evaluate the performance of proposed LMAO-PU by comparing it with six representative PU learning algorithms, and the comparison results on 10 high-dimensional PU datasets justify the superiority of LMAO-PU in obtaining the sparse PU classifier with higher quality. In addition, the extra experiments also verify the effectiveness of the suggested alternating optimization and the loss matrix.

The rest of this paper is organized as follows. In Section 2, the preliminaries about sparse PU learning and the related works about PU learning are reviewed. The detailed procedure of the proposed algorithm is elaborated in Section 3, and the experimental results are reported in Section 4. Finally, the conclusions and some future remarks are drawn in Section 5.

2. Preliminaries and related work

In this section, we first introduce some preliminaries about sparse PU learning. Then, the related research work about PU learning is reviewed in detail.

2.1. Sparse PU learning

PU learning has been arisen in many real applications, such as bioinformatics [21], recommended systems [1] and drug discovery [3] etc, which is viewed as a special case of supervised PN learning problem. During the training phase of PU classifiers, only P samples and U samples are available. Thus, the goal of PU learning is to train a binary classifier from the PU data, which can well predict the labels of unseen samples [5]. Sparse PU learning is regarded as a variation of PU learning, aiming to obtain a sparse PU classifier from high-dimensional PU data, which is defined as follows:

Given a high-dimensional PU dataset $X_{PU} = \{(x_i, y_i)\}_{i=1}^n$, where $x_i \in \mathbb{R}^d$ is the i th PU sample with d dimensions, y_i represents that the label of x_i is positive ($y_i = 1$) or uncertain ($y_i = 0$) and n is the number of PU samples. Let Ω be the search space composed of d -dimensional vector $D_d = \{f_1, \dots, f_d\}$, where $f_1, \dots, f_d$ are d components of samples. Sparse PU learning is to learn a sparse classification function $C_{\mathbb{R}^{D_m}}(x)$ from m dimensions sparse subspace $\mathbb{R}^{D_m}$ ($\mathbb{R}^{D_m} \subset \Omega$), where $C_{\mathbb{R}^{D_m}}(\cdot)$ denotes the trained sparse PU classifier, x represents the unseen sample to be predicted and $m < d$.

2.2. Related work

Different from the traditional supervised learning, in PU learning, besides positive samples, no labeled negative samples are available for training PU classifiers. To this end, in recent years, PU learning has been widely studied and a variety of competitive PU algorithms have been developed, which can be mainly divided into three categories.

The first category is sample selection-based approaches, where only a part of U samples are selected and regarded as the “reliable” negative samples for training PU classifier. For example, Li et al. [7] proposed a sample selection-based approach, named Roc-SVM, for PU text classification, where a two-step strategy was suggested to train the PU classifier. Specifically, the first step is to select some “reliable” unlabeled samples as the negative ones and the second step is to apply support vector machine (SVM) [22] to build a PU classifier for classification. Based on the suggested two-step strategy, Liu et al. [8] further designed an improved SVM technique to enhance the performance of the obtained PU classifier. Carnevali et al. [23] built a similarity graph to identify “reliable” negative samples from unlabeled data. Luo et al. [9] proposed an effective negative sample selector as the agent responsible for selecting negative samples from the unlabeled data. These approaches mentioned above have made good effectiveness in solving PU learning problem. However, since only consider the part of U samples and the remaining unlabeled samples are not utilized, therefore, the improvement of the performance of the PU classifiers is limited [24].

The second category is biased-based learning approaches, where all U samples are treated as the negative ones with label noise [25,26]. Then, the P samples and the noisy N samples are used for building PU classifiers. For example, Lee et al. [27] transformed PU learning into the problem of learning from the noisy negative samples, in which the logistic regression with convex lost function was adopted for the weighted samples to handle the issue of noises. Due to the inaccuracy of the weight adjustment, Shi et al. [11] suggested to directly decompose the empirical loss on the noisy negative data and reduce the influence of noises by estimating the centroid of noisy negative samples. Northcutt et al. [28] proposed a novel algorithm, named CL, to directly estimate the joint distribution between noisy labels and unknown labels, aiming to find the label errors exactly, which can be used to increase the PU classification model accuracy. However, when a large number of potential positive samples are in the U samples, since they are mistakenly regarded as negative ones, the performance of these approaches can be significantly degraded [29].

The third category is class prior-based approaches, where each U sample is regarded as positive or negative. With the converted PU data, PU learning is transformed into a cost-sensitive learning problem. For example, Plessis et al. [30] proposed a non-convex loss function (i.e., ramp loss function) for the converted cost-sensitive learning problem. Since the loss function is non-convex, it is difficult to obtain the global optimal solution as the final classifier. To this end, Plessis et al. [14] further discussed different loss functions and proposed a convex double hinge loss function for PU learning, named uPU, which can obtain the PU classifier with better classification performance. Recently, Chen et al. [31] imposed different misclassification costs on different classes and employed double hinge loss to build for the proposed algorithm under the framework of empirical risk minimization. Zhang et al. [32] assigned a set of candidate labels (i.e., positive and negative) to each U sample and proposed a novel PU learning algorithm, termed PULD, in which a disambiguation technique was designed to determine the true label of each U sample. The experimental results have demonstrated the superiority of PULD over the state-of-the-art PU learning algorithms. In these PU algorithms aforementioned, the proportion of positive samples in U samples is needed as a prior information, which is difficult to meet in real PU applications. Besides the algorithms tailored for PU learning, there also exists a general algorithm, named PNU [33], for PU learning, which regards PU learning

Table 1
The pros and cons of existing PU algorithms.

Category	Do they need prior knowledge of U data?	Do they need continuous and differentiable objective functions?	Can they tackle high-dimensional PU data?
Sample selection based PU algorithms [7–9,22–24]	N	Y	N
Biased learning based PU algorithms [11,25–29]	N	Y	N
Class prior-based PU algorithms [14,30–35]	Y	Y	N
The other PU algorithm [17]	N	N	N

as a special case of PNU (positive, negative and unlabeled) learning. To this end, a novel semi-supervised learning approach is designed for PU learning. Experimental results demonstrated the suggested PNU algorithm can well solve PU learning in terms of classification accuracy. More class prior-based approaches can be found in [34,35].

Recently, evolutionary computation (EC) methods, particularly, MOEAs have gained much focus, because MOEAs require no domain knowledge and assumptions about the data and optimized objectives. The first MOEA-based PU algorithm, named MOEA-PUL was proposed by Qiu et al. [17], in which PU learning was regarded as a multi-objective optimization problem and an MOEA was designed to solve it. Experimental results have shown that compared with the state-of-the-art PU algorithms, the proposed MOEA-PUL could obtain the PU classifiers with higher qualities.

For readers to quickly catch the related work of PU learning, we briefly summarize the pros and cons of the existing PU algorithms mentioned above and shown in Table 1. From Table 1, we can find that all of the existing PU algorithms are designed for traditional PU learning, aiming to train PU classifiers with high quality. Few of them consider the PU learning from high-dimensional PU data, since labels uncertainty (maybe positive or negative) brings the difficulty and challenge for the existing PU algorithms. To this end, in this paper, we proposed a two-stage alternating optimization method for sparse PU learning under the high-dimensional data situation. Different from the former three categories PU algorithms, the recently suggested MOEA-PUL [17] has shown the state-of-the-art performance without any prior knowledge and assumptions. Thus, we preserve the pros of MOEAs and use MOEAs to solve the suggested two optimization models, aiming to achieve more competitive sparse PU classifiers.

Fig. 1 gives an example to illustrate the challenges of sparse PU learning. In this figure, a nine-dimensional (D_9) PU dataset X_{PU} is given, where $\{X_U^1, \dots, X_U^{10}\}$ are 10 unlabeled samples and $\{X_P^1, \dots, X_P^5\}$ are 5 positive samples. Due to the labels uncertainty of U samples, we consider two possible cases in the following. Case 1: $\{X_U^1, \dots, X_U^5\}$ are assumed as positive samples and $\{X_U^6, \dots, X_U^{10}\}$ are assumed as negative samples. Case 2: $\{X_U^1, \dots, X_U^5\}$ are treated as negative while $\{X_U^6, \dots, X_U^{10}\}$ are regarded as positive. Then, feature selection methods, e.g. [36], can be used to obtain the useful features, with which the sparse subspace could be achieved. From this figure, we can find that for Case 1, $\{f_1, f_3, f_5, f_7, f_9\}$ are selected to consist of the sparse subspace $\mathbb{R}^{D_5}$, with which to construct sparse PU model $C_{\mathbb{R}^{D_5}}$. While for Case 2, the sparse PU model $C_{\mathbb{R}^{D_4}}$ is constructed on the sparse subspace $\mathbb{R}^{D_4}$ using the useful features $\{f_2, f_4, f_6, f_8\}$. From the two sparse PU models $C_{\mathbb{R}^{D_5}}$ and $C_{\mathbb{R}^{D_4}}$, we can find that the final achieved two sparse PU models must be different. In addition, from this example, we also note that this challenge is from the labels uncertainty of U samples. To tackle the challenge, in this paper, we design an alternating optimization method, where PU model and sparse model are optimized alternately to obtain sparse PU models with high quality.

3. The proposed LMAO-PU

In this section, we will present the details of the proposed alternating optimization method for the problem of sparse PU learning, which

is composed of two stages. Specifically, the first stage is to design a PU model to obtain an optimal PU classifier. With the optimal PU classifier achieved in the first stage, the second stage is to build a sparse model to find the optimal sparse classifier. The optimal sparse PU classifier is then fed into the first stage to further enhance its classification performance. The two stages are performed alternately for PU learning from high-dimensional PU data. Fig. 2 gives an illustration of the proposed LMAO-PU. As can be seen from this figure, the loss matrix is a key metric designed for evaluating the performance of PU classifier on the U samples and used to guide the suggested alternating alternative optimization, which will be elaborated in the following part.

3.1. Loss matrix

As discussed above, a two-stage alternating optimization is designed for sparse PU learning, in which the loss matrix, **LM**, is designed to evaluate the classification performance of the PU classifier on U samples. The basic idea is to utilize the labels of the neighbor samples of each U sample to construct a classifier to calibrate the label of the U sample predicted by a PU classifier, which is from the following intuition. For the neighbor samples of each U sample, the larger number of P samples in them, the higher probability of the U sample being labeled as positive. With the label information of the neighbor samples, the label of the U sample predicted by a given PU classifier is calibrated and a novel metric, named the LRU (loss rate on the unlabeled) is proposed to evaluate the performance of the PU classifier on the U samples. Specifically, given N PU classifiers and $|X_U|$ U samples, the loss matrix **LM** is defined as follows and generated by the following two steps, in which each element $\mathbf{LM}(i, j)$ represents the loss rate of the i th ($i = 1, \dots, N$) PU classifier on the j th ($j = 1 \dots |X_U|$) U sample.

The first step is to achieve the neighbor label information of each U sample. To be specific, for the j th U sample, X_U^j , the k neighbor samples are selected and used to construct a classifier by the k-Nearest Neighbor (kNN) algorithm [37]. In the selected k neighbor samples, the U samples are treated as negative ones, aiming to train a kNN classifier. Let $Num^P(X_U^j)$ and $Num^U(X_U^j)$ respectively be the number of P samples and U samples in the k neighbor samples. If the number of P samples is larger than that of U samples, the probability of the j th U sample being labeled as positive is $Num^P(X_U^j)/k$. Otherwise, the j th U sample is labeled as negative with a probability of $Num^U(X_U^j)/k$. Using the probability of being labeled as positive or negative for each U sample, the second step is to calculate the loss rate of the PU classifier on each U sample, aiming to approximatively evaluate the performance of the PU classifier. Specifically, let $C_i(\cdot)$ be the i th PU classifier used to calculate the prediction score and $sign : \mathbb{R} \rightarrow \{-1, 1\}$ be the map function used to achieve the predicted label $sign(C_i(X_U^j))$ is inconsistent with the most of neighboring labels, the loss rate $\mathbf{LM}(i, j)$ is defined as the probability of the j th U sample being labeled as positive (or negative). Otherwise, the loss rate of $\mathbf{LM}(i, j)$ is denoted as 0. The loss rate of $\mathbf{LM}(i, j)$ is shown in Fig. 3. Algorithm 1 shows the main process of calculating the loss matrix **LM**.

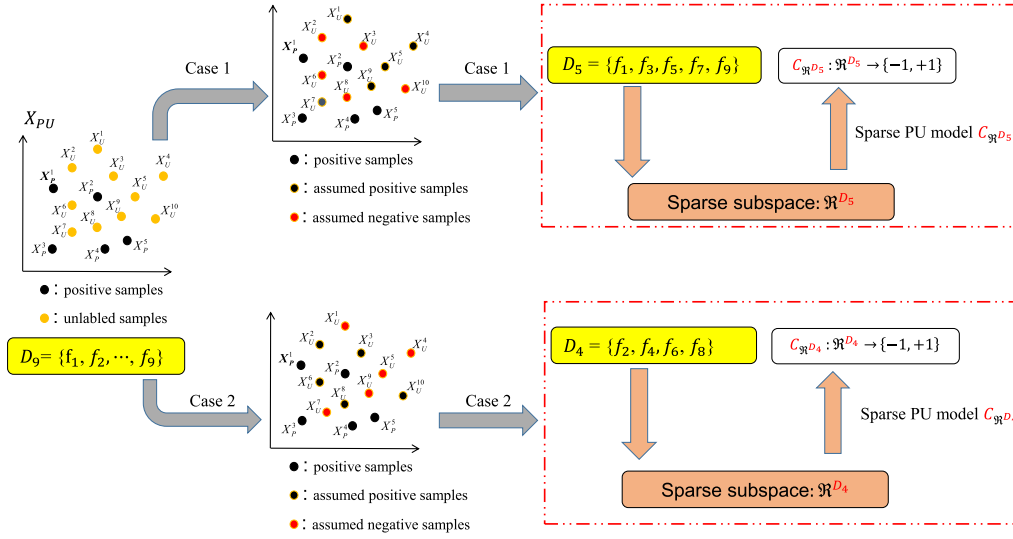


Fig. 1. An example to illustrate the challenge of sparse PU learning.

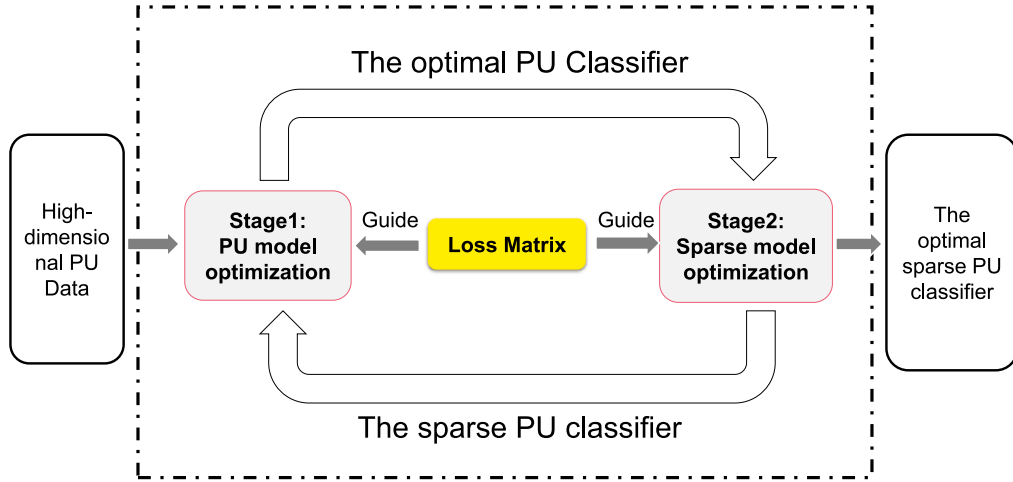


Fig. 2. An illustration of the proposed LMAO-PU.

	$NUM^P(X_U^j) > NUM^U(X_U^j)$	$NUM^P(X_U^j) \leq NUM^U(X_U^j)$
$sign(C_i(X_U^j)) < 0$	$NUM^P(X_U^j) / k$	0
$sign(C_i(X_U^j)) \geq 0$	0	$NUM^U(X_U^j) / k$

$$\mathbf{LM}(i, j)$$
Fig. 3. $\mathbf{LM}(i, j)$: the loss rate of the i th PU classifier $C_i(\cdot)$ on the j th U sample X_U^j .

Fig. 4 gives an example to illustrate the process of calculating the loss matrix $\mathbf{LM}$, in which there are five P samples (X_P^1 to X_P^5), five U samples (X_U^6 to X_U^{10}) and four PU classifiers ($C_1(\cdot)$ to $C_4(\cdot)$). For each U sample, k ($k = 5$) neighbor samples are selected to construct a kNN classifier. Specifically, in Fig. 4(a), for each U sample, five neighbor samples are selected, where '1' denotes the selected P sample,

'0' represents the selected U sample and '-' denotes the sample that is not selected as the neighbor sample. Taking X_U^6 as an example, $\{X_P^1, X_P^5, X_U^7, X_U^8, X_U^9\}$ are selected as its neighbor samples, in which $\{X_U^7, X_U^8, X_U^9\}$ are regarded as negative samples. For each U sample, Fig. 4(b) gives the ratio of positive samples and unlabeled samples in the k neighbor samples, respectively. For X_U^6 , since there exists two P

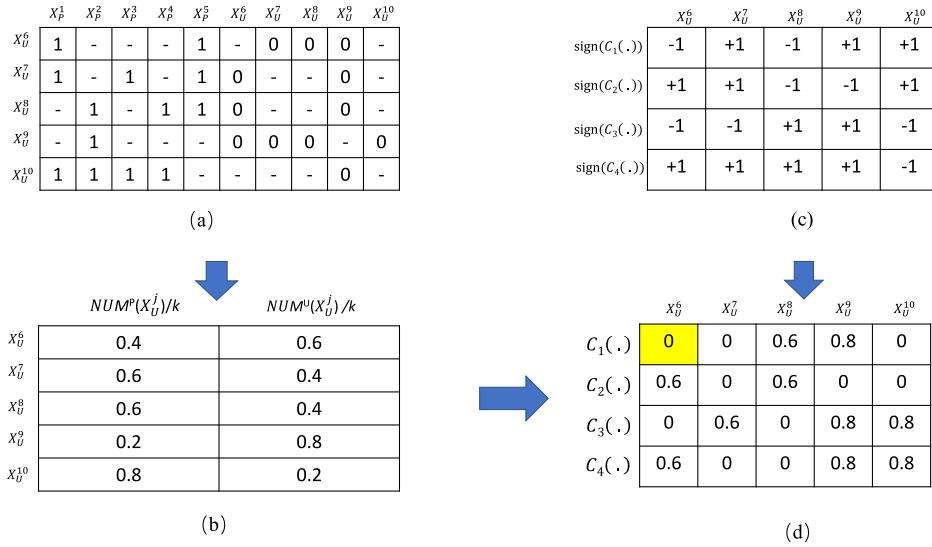


Fig. 4. An illustrative example for calculating the loss matrix $\mathbf{LM}$. (a) the selected k neighbor samples of each U sample. (b) the probability of each U sample being labeled as positive or negative. (c) the predicted labels of five U samples using the PU classifiers $C_1(\cdot)$ to $C_4(\cdot)$. (d) the generated loss matrix $\mathbf{LM}$.

Algorithm 1: LossMatrix

Input: X_{PU} : the PU dataset, X_U : the set of U samples, k : the neighbor size for each U sample, P_M : the set of PU classifiers, N : the number of PU classifiers;

Output: $\mathbf{LM}$: the loss matrix;

- 1: $\mathbf{LM} \leftarrow \text{zeros}(N, |X_U|)$;
- 2: **for** $i = 1$ to N **do**
- 3: **for** $j = 1$ to $|X_U|$ **do**
- 4: $C_i(\cdot) \leftarrow P_M^i$; // The i -th PU classifier.
- 5: Calculate the predicted label $sign(C_i(X_U^j))$;
- 6: $X_{PU}^{j_1, \dots, j_k} \leftarrow$ Choose k neighbor samples of the U sample X_U^j ;
- 7: Calculate $\mathbf{LM}(i, j)$ according to Fig. 3;
- 8: **end for**
- 9: **end for**

sample and three U samples (regarded as N samples) in its neighbor samples, the probability of it being labeled as positive or negative is $2/5$ and $3/5$ respectively. Fig. 4(c) presents the predicted labels of five U samples (X_U^6 to X_U^{10}) using the PU classifiers $C_1(\cdot)$ to $C_4(\cdot)$, where ‘+1’ and ‘-1’ denote the predicted positive and negative class, respectively. For the PU classifier $C_1(\cdot)$, it predicts the U samples X_U^6 and X_U^8 as the negative ones while the U samples X_U^7 , X_U^9 and X_U^{10} are predicted as positive ones. Finally, with the results of Fig. 4(b) and (c), (d) gives the loss matrix $\mathbf{LM}$ with four rows and five columns. For example, in $\mathbf{LM}$, $C_1(\cdot)$ predicts the label of X_U^6 as negative and the label determined by its five neighbor samples is also negative. Thus, the value of the element $\mathbf{LM}[1][1]$ is set to 0, which is denoted as yellow in this figure.

3.2. The proposed alternating optimization method

With the designed loss matrix, an alternating optimization method is proposed for sparse PU learning. Fig. 5 presents the procedure of the proposed alternating optimization, which consists of two stages. As can be seen from this figure, in the first stage (Stage 1), a PU model is built to find the optimal PU classifier. In the second stage (Stage 2), with the obtained optimal PU classifier, a sparse model is constructed to find its sparse PU classifier with high quality. The two stages are performed alternately and together used for obtaining the optimal sparse PU classifier, which is elaborated as follows.

3.2.1. Stage 1: PU model optimization

Based on the designed loss matrix $\mathbf{LM}$, a novel metric LRU , together with the metric of TPR , is used to construct the PU model to find the optimal PU classifier. To this end, we formulate it as a bi-objective optimization problem, which is presented as follows.

$$MOP1 : f(X_P, X_U) = (f_{TPR}(X_P), f_{LRU}(X_U))$$

$$\begin{aligned} \text{Maximize} : f_{TPR}(X_P) &= \sum_{i=1}^{|X_P|} I(\text{sign}(C_i(X_P^i)) = 1) / |X_P| \\ \text{Minimize} : f_{LRU}(X_U) &= \sum_{j=1}^{|X_U|} \mathbf{LM}[i][j] / (|X_U|) \end{aligned} \quad (1)$$

where X_P and X_U respectively denote the set of P samples and the set of U samples, X_P^i is the i th positive sample in X_P , $|\cdot|$ represents the number of samples for a given set and $C_i(\cdot)$ denotes the i th PU classifier. $\mathbf{LM}[i][j]$ is the loss rate of the i th PU classifier $C_i(\cdot)$ on the j th U sample X_U^j , which is used to evaluate the performance of the PU classifier on the j th U sample. $I(\cdot)$ is the indicator function, $I(\cdot) = 1$ if the argument is true, and $I(\cdot) = 0$ otherwise.

The first objective $f_{TPR}(X_P)$ is a widely used metric in PN learning, which is used to measure the performance of the given PU classifier on the X_P samples. Maximizing the objective $f_{TPR}(X_P)$ is expected to achieve the PU classifier with the highest classification accuracy on the P samples. The second objective $f_{LRU}(X_U)$ represents the inconsistency between the predicted label and the labels of the most of its neighbors, which is used to evaluate the performance of the PU classifier $C_i(\cdot)$ on the U samples. The basic idea behind this is that the predicted labels by PU classifiers should be as same with the label determined by its neighbor samples as possible. Thus, minimizing $f_{LRU}(X_U)$ is to make the predicted label by the PU classifier be consistent with the labels of most of its neighbor samples.

For the bi-objective optimization problem mentioned above, we employ the evolutionary computation (EC) paradigm [38,39] with well global search ability to solve it. To this end, an initial population P_M with N individuals¹ is generated, where each individual represents a PU classifier and denoted as $Ind_w = \{w_1, w_2, \dots, w_d\}$, where $w_i \in \mathbb{R}$ is the weight of the i th dimension of the PU classifier Ind_w and d is the dimension of samples.

¹ In this paper, the ‘ N ’ denotes the number of PU classifiers or the size of population, the ‘ P_M ’ represents the set of PU classifiers or the PU model population.

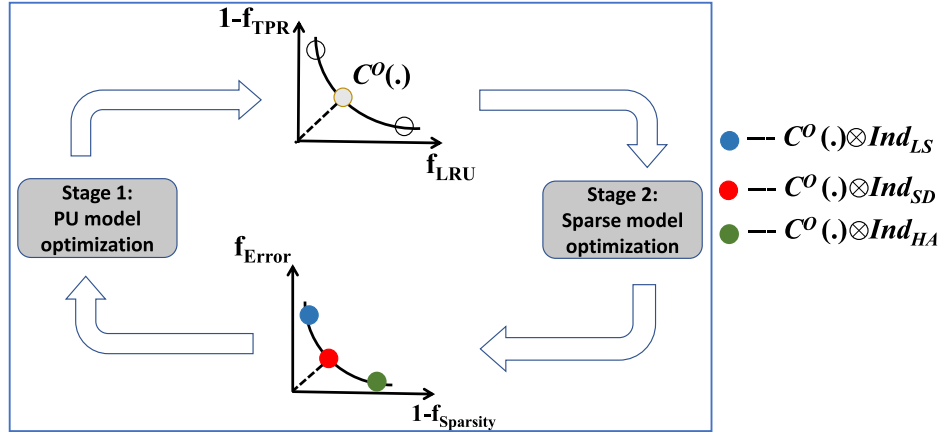


Fig. 5. The process of the proposed alternating optimization for sparse PU learning. $C^O(\cdot)$ denotes the optimal PU classifier obtained in Stage 1. $C^O(\cdot) \otimes Ind_{LS}$, $C^O(\cdot) \otimes Ind_{SD}$ and $C^O(\cdot) \otimes Ind_{HA}$ respectively represent three special sparse PU classifier obtained in Stage 2 based on the optimal PU classifier $C^O(\cdot)$.

Based on the adopted encoding mentioned above, a classical multi-objective evolution algorithm, NSGA-II [40], is adopted for the suggested bi-objective optimization problem *MOP1*. Algorithm 2 presents the process of PU model optimization, which is composed of three main components. Firstly, after the individuals in P_M are evaluated (Line 3), a binary tournament mechanism is adopted to select individuals for producing the mating pool (Line 4). Specifically, two individuals are randomly chosen from the parent population. If one solution dominates the other solution, then the former solution is chosen. If the two solutions are nondominated with each other, then one of them will be randomly chosen for reproduction. Secondly, the simulated binary crossover [41] and polynomial mutation [42] are used to generate N offsprings from the current population (Line 5). Finally, N individuals are selected as the parent population survived to the next generation (Line 6). The first to third steps repeat until the termination condition is fulfilled. After the generation being terminated, we can obtain a set of non-dominated solutions $NonS$ according to the non-dominated sorting method [40]. The solution with the minimum distance with the original point is selected as the optimal PU classifier, denoted as $C^O(\cdot)$, which shows better performance in terms of *TPR* and *LRU* and fed into the stage of sparse model optimization (Stage 2).

Algorithm 2: PUModelOptimize

Input: X_p : the set of positive samples, X_U : the set of unlabeled samples, LM : loss matrix, P_M : the PU model population, N : the population size, gen : the number of iteration in the PU model optimization;
Output: P_M : the PU model population; $C^O(\cdot)$: the optimal PU classifier.
1: $iter = 1$;
2: **while** $iter < gen$ **do**
3: Evaluate the individuals in population P_M according to *MOP1* (1);
4: $P'_M \leftarrow \text{Mating_selection}(P_M, N)$;
5: $P_M \leftarrow P_M \cup \text{Variation}(P'_M, N)$;
6: $P_M \leftarrow \text{Environmental_selection}(P_M, N)$;
7: $iter = iter + 1$;
8: **end while**
9: $NonS \leftarrow \text{SelectNonDominatedsets}(P_M)$;
10: $C^O(\cdot) \leftarrow \text{Select an optimal PU classifier from the nondominated solution set } NonS$;

3.2.2. Stage 2: Sparse model optimization

In the first stage, the optimal PU classifier $C^O(\cdot)$ is obtained by optimizing the proposed PU model. Based on the optimal PU classifier, a sparse model is constructed with the aim to find the sparse PU classifier with high quality, which is formulated as another bi-objective

optimization problem and described as follows.

$$\begin{aligned}
MOP2 : F(Ind_s) &= (f_{Sparsity}(Ind_s), f_{Error}(Ind_s)) \\
Maximize \quad f_{Sparsity}(Ind_s) &= 1 - \frac{|\{Ind_s == 1\}|}{d} \\
Minimize \quad f_{Error}(Ind_s) &= \frac{\sum_{i=1}^{|X_p|} I(\text{sign}(\widetilde{C^O}(X_p^i)) \neq 1) + \sum_{j=1}^{|X_U|} LM[o][\widetilde{X_U^j}]}{|X_{PU}|}
\end{aligned} \quad (2)$$

where Ind_s is a binary vector with d components, in which each bit denotes the corresponding weight in the PU classifier $C^O(\cdot)$ whether to be selected ('1' is selected and '0' otherwise), X_p is the set of P sample, X_p^i is the i th positive sample and X_U are the set of U samples. $\widetilde{C^O}(\cdot) = C^O(\cdot) \otimes Ind_s$ denotes a sparse PU classifier, where $C^O(\cdot)$ is the optimal PU classifier achieved in the first stage and ' $\otimes$ ' represents the element-wise multiplication operation. $LM[o][\widetilde{X_U^j}]$ denotes the loss rate of the optimal PU classifier $C^O(\cdot)$ on the j th sparse unlabeled sample $\widetilde{X_U^j}$, which equals to measure the performance of the sparse PU classifier $C^O(\cdot) \otimes Ind_s$ on the j th U sample X_U^j .

The first objective $f_{Sparsity}(Ind_s)$ is used to measure the sparsity of the sparse PU classifier, which is from the following intuition. In the high-dimensional PU data, the existing redundant and irrelative features degrade the performance of the obtained PU classifier. Thus, we hope to find a sparse PU classifier (i.e., some unimportant bits in the classifier are set to 0) from high-dimensional PU data. Maximizing the $f_{Sparsity}(Ind_s)$ is to guarantee the high sparsity of the obtained PU classifier. The second objective $f_{Error}(Ind_s)$ is to evaluate the quality of the sparse PU classifier on P and U samples, respectively. Minimizing the $f_{Error}(Ind_s)$ is to make the error of the obtained sparse PU classifier on PU data minimization.

To solve the suggested bi-objective optimization problem *MOP2*, a sparse population P_S with N individuals are generated, where each individual is denoted as a binary vector and used to represent a candidate sparse PU classifier. Similarly, we also adopt NSGA-II to solve the suggested bi-objective optimization problem for sparse model optimization, which is presented in Algorithm 3.

After the evolution is finished (Lines 1–8), a set of non-dominated solutions $NonSP$ are achieved from the population P_S , where each one can be used to denote a sparse representation of the optimal PU classifier achieved in the first stage. From these solutions, three binary vectors (Ind_{HA} , Ind_{LS} , Ind_{SD}) are selected, which respectively denote three special sparse PU classifiers, that is, the sparse PU classifier ($C^O(\cdot) \otimes Ind_{HA}$) with highest accuracy of classification, the sparse PU classifier ($C^O(\cdot) \otimes Ind_{LS}$) with largest degree of sparsity and the sparse PU classifier ($C^O(\cdot) \otimes Ind_{SD}$) with the smallest distance between the individual Ind_{SD} to the origin point. With the achieved binary vectors,

the sparse PU dataset $\widetilde{X}_{PU}^2$ and the sparse PU model population³ are obtained.

Algorithm 3: SparseModelOptimize

Input: X_P : the set of positive samples, X_U : the set of unlabeled samples, LM : loss matrix, d : the number of dimensions, $C^O(\cdot)$: the optimal PU classifier obtained in the first stage, P_M : the PU model population, P_S : the sparse population, N : the population size, gen : the number of iteration in the sparse model optimization;
Output: $\widetilde{X}_{PU}$: the sparse PU dataset, P_M : the sparse PU model population;
1: $iter = 1$;
2: **while** $iter < gen$ **do**
3: Evaluate the individuals in population P_S according to MOP2 (2);
4: $P'_S \leftarrow \text{Mating_selection}(P_S, N)$;
5: $P_S \leftarrow P_S \cup \text{Variation}(P'_S, N)$;
6: $P_S \leftarrow \text{Environmental_selection}(P_S, N)$;
7: $iter \leftarrow iter + 1$;
8: **end while**
9: $NonSP \leftarrow \text{SelectNonDominatedsets}(P_S)$;
10: $[Ind_{HA}, Ind_{LS}, Ind_{SD}] \leftarrow \text{Select three individuals from the non-dominated solution set } NonSP$;
11: Update PU dataset to obtain the sparse $\widetilde{X}_{PU}^2$;
12: $P_M \leftarrow P_M \otimes \{Ind_{HA}, Ind_{LS}, Ind_{SD}\}$;
//Obtain the sparse PU model population P_M^3 by random selecting one from the three individuals to perform element-wise multiplication operation;

Algorithm 4: LMAO-PU

Input: X_{PU} : PU dataset, X_P : the set of positive samples, X_U : the set of unlabeled samples, d : the number of dimensions, $Maxiter$: the maximum number of the iteration, gen : the number of iteration in the PU model optimization and sparse model optimization stages, k : the neighbor size for each unlabeled sample, N : the population size;
Output: The final set of non-dominated solution FS ;
1: $P_M \leftarrow \text{Initialize the PU model population}$;
2: $P_S \leftarrow \text{Initialize the sparse population}$;
3: $LM \leftarrow \text{LossMatrix}(X_{PU}, X_U, k, P_M, N)$;
//The proposed Algorithm 1
4: $iter = 1$;
5: **while** $iter < Maxiter$ **do**
6: $[P_M, C^O(\cdot)] \leftarrow \text{PUModelOptimize}(X_P, X_U, LM, P_M, N, gen)$; //The proposed Algorithm 2
7: $[\widetilde{X}_{PU}, P_M] \leftarrow \text{SparseModelOptimize}(X_P, X_U, LM, d, C^O(\cdot), P_M, P_S, N, gen)$; //The proposed Algorithm 3
8: $LM \leftarrow \text{LossMatrix}(\widetilde{X}_{PU}, X_U, k, P_M, N)$;
9: $iter = iter + 1$;
10: **end while**
11: $FS \leftarrow \text{SelectNonDominatedset}(P_M)$;
//Select the non-dominated solutions from P_M

3.2.3. The framework of LMAO-PU

The framework of the proposed LMAO-PU is presented in Algorithm 4, which is composed of the following three steps. In the first step, two populations (P_M and P_S), respectively denoting N PU classifiers and N binary vectors, are initialized (Line 1 and Line 2). Then, the loss matrix LM is initialized (Line 3) by the proposed Algorithm 1. The second step is the main loop, in which the suggested PU model and sparse model are optimized alternately with the aim to obtain the optimal sparse PU classifier. Specifically, the PU model is firstly optimized

² The sparse PU dataset, $\widetilde{X}_{PU} \leftarrow X_{PU} \otimes (Ind_{HA} * ones(|X_{PU}|, d))$, where $ones(|X_{PU}|, d)$ denotes a matrix with $|X_{PU}|$ rows and d columns and all elements are one).

³ The sparse PU model population P_M is obtained by random selecting one from the three binary vectors to perform element-wise multiplication operation (Line 10 and 11 in Algorithm 3).

Table 2

10 datasets with different characteristics.

Dataset	Dimension	Sample	Num of P	Num of N
LSVT	310	126	42	84
Madelon	500	2600	1300	1300
Gse72526	800	61	26	35
Colon	2000	62	22	40
Lymphoma	4026	96	46	50
DLBCL	5269	77	19	58
Brain1	5920	90	30	60
Drivface	6400	573	546	27
Prostate	10,509	102	50	52
GLI_85	22,283	85	26	59

by the suggested Algorithm 2 to obtain the optimal PU classifier by addressing the challenge of the label uncertainty of the U samples (Line 6). Then, with the obtained optimal PU classifier, the sparse model is optimized by the suggested Algorithm 3, to achieve the optimal sparse PU classifier (Line 7). At the end of the second step, the loss matrix is updated with the achieved sparse PU dataset² and sparse PU model population³ (Line 8). The two models are optimized alternately for sparse PU learning until the terminate condition is fulfilled. In the third step, from the final model population P_M , the non-dominated sorting method [40] can be employed to select the non-dominated solutions as the final sparse PU classifiers.

4. Experimental results

In this section, we empirically verify the performance of the proposed LMAO-PU for sparse PU learning by comparing it with six state-of-the-art PU learning algorithms on 10 high-dimensional datasets, whose dimensions range from 310 to 22,283. To be specific, the experimental settings, including the PU datasets, the comparison algorithms and the parameter settings, are firstly presented. Then, the empirical results between LMAO-PU and six baselines are reported. Thirdly, the effectiveness of the proposed two-stage alternating optimization and the loss matrix are discussed. Finally, the sensitivity of parameters k and gen in the proposed method are analyzed.

4.1. Datasets

In our experiments, 10 real-world binary datasets with different characteristics are adopted to evaluate the performance of the proposed LMAO-PU, which are publicly available on the data repository,⁴ The detailed statistical information of these datasets are summarized in Table 2.

For each dataset, a widely used PU data preprocessing approach is employed to generate the experimental PU dataset [32]. Specifically, we randomly select $\mu\%$ ($\mu \in \{20, 30, 40\}$) positive samples from the known positive sample set, and combined with the negative samples to generate the U samples in PU data. The rest of positive samples are viewed as the P samples in PU data. For each dataset, five-fold cross validation [43] to partition the datasets, with which to evaluate the performance of different algorithms, including ours. Specifically, each dataset is partitioned into five folds, in which four folds are used as the training datasets for evaluating the performance of individuals during the stage of training and the remained one fold is used as the test datasets for testing the classification accuracy of the obtained PU classifier. The process of training-testing is repeated five runs independently and the average classification accuracy is reported.

⁴ <https://archive.ics.uci.edu/ml/> <http://www.gemssystem.org>.

Table 3

Average sparsity of LMAO-PU vs. CL, uPU, PNU, PULD, BAUC-OF and MOEA-PUL algorithms on the 10 datasets.

Dataset	$\mu\%$	CL [28]	uPU [14]	PNU [33]	PULD [32]	BAUC-OF [44]	MOEA-PUL [17]	LMAO-PU
LSVT	0.2	0.512 $\pm$ 0.062($\uparrow$ 83.9%)	0.108 $\pm$ 0.045($\uparrow$ 771.64%)	0.074 $\pm$ 0.014($\uparrow$ 1180.7%)	0.173 $\pm$ 0.07($\uparrow$ 443.63%)	~	0.333 $\pm$ 0.095($\uparrow$ 182.71%)	0.942 $\pm$ 0.061
LSVT	0.3	0.434 $\pm$ 0.079($\uparrow$ 109.83%)	0.094 $\pm$ 0.033($\uparrow$ 869.07%)	0.081 $\pm$ 0.036($\uparrow$ 1019.05%)	0.145 $\pm$ 0.076($\uparrow$ 529.08%)	~	0.233 $\pm$ 0.127($\uparrow$ 289.88%)	0.910 $\pm$ 0.137
LSVT	0.4	0.443 $\pm$ 0.089($\uparrow$ 109.91%)	0.107 $\pm$ 0.042($\uparrow$ 765.47%)	0.135 $\pm$ 0.088($\uparrow$ 589.47%)	0.168 $\pm$ 0.066($\uparrow$ 453%)	~	0.208 $\pm$ 0.085($\uparrow$ 347.89%)	0.930 $\pm$ 0.086
Madelon	0.2	0.342 $\pm$ 0.102($\uparrow$ 189.11%)	0.055 $\pm$ 0.009($\uparrow$ 1689.49%)	0.151 $\pm$ 0.096($\uparrow$ 553.31%)	0.137 $\pm$ 0.053($\uparrow$ 619.64%)	~	0.217 $\pm$ 0.04($\uparrow$ 354.32%)	0.988 $\pm$ 0.014
Madelon	0.3	0.249 $\pm$ 0.102($\uparrow$ 286.39%)	0.039 $\pm$ 0.012($\uparrow$ 2338.58%)	0.21 $\pm$ 0.172($\uparrow$ 358.4%)	0.118 $\pm$ 0.028($\uparrow$ 712.3%)	~	0.257 $\pm$ 0.084($\uparrow$ 274.41%)	0.961 $\pm$ 0.080
Madelon	0.4	0.346 $\pm$ 0.019($\uparrow$ 184.17%)	0.045 $\pm$ 0.008($\uparrow$ 2101.79%)	0.158 $\pm$ 0.138($\uparrow$ 519.95%)	0.07 $\pm$ 0.005($\uparrow$ 1300.26%)	~	0.232 $\pm$ 0.069($\uparrow$ 323.29%)	0.982 $\pm$ 0.020
Gse72526	0.2	0.376 $\pm$ 0.061($\uparrow$ 155.27%)	0.084 $\pm$ 0.011($\uparrow$ 1047.16%)	0.089 $\pm$ 0.012($\uparrow$ 976.47%)	0.152 $\pm$ 0.041($\uparrow$ 530.39%)	~	0.578 $\pm$ 0.047($\uparrow$ 66.16%)	0.961 $\pm$ 0.093
Gse72526	0.3	0.357 $\pm$ 0.029($\uparrow$ 177.67%)	0.086 $\pm$ 0.011($\uparrow$ 1052.18%)	0.081 $\pm$ 0.011($\uparrow$ 1127.09%)	0.118 $\pm$ 0.034($\uparrow$ 736.7%)	~	0.595 $\pm$ 0.043($\uparrow$ 66.55%)	0.991 $\pm$ 0.007
Gse72526	0.4	0.395 $\pm$ 0.065($\uparrow$ 151.81%)	0.087 $\pm$ 0.009($\uparrow$ 1040.4%)	0.081 $\pm$ 0.011($\uparrow$ 1128.4%)	0.111 $\pm$ 0.027($\uparrow$ 798.98%)	~	0.362 $\pm$ 0.062($\uparrow$ 174.76%)	0.995 $\pm$ 0.003
Colon	0.2	0.24 $\pm$ 0.116($\uparrow$ 274.42%)	0.028 $\pm$ 0.002($\uparrow$ 3053.95%)	0.031 $\pm$ 0.003($\uparrow$ 2822.8%)	0.036 $\pm$ 0.004($\uparrow$ 2372.52%)	~	0.597 $\pm$ 0.032($\uparrow$ 50.18%)	0.897 $\pm$ 0.108
Colon	0.3	0.313 $\pm$ 0.046($\uparrow$ 205.08%)	0.028 $\pm$ 0.003($\uparrow$ 3333.09%)	0.027 $\pm$ 0.003($\uparrow$ 3395.97%)	0.041 $\pm$ 0.008($\uparrow$ 2239.8%)	~	0.577 $\pm$ 0.056($\uparrow$ 65.41%)	0.954 $\pm$ 0.092
Colon	0.4	0.269 $\pm$ 0.085($\uparrow$ 269.08%)	0.032 $\pm$ 0.003($\uparrow$ 2982.89%)	0.035 $\pm$ 0.006($\uparrow$ 2739.97%)	0.037 $\pm$ 0.009($\uparrow$ 2601.36%)	~	0.647 $\pm$ 0.12($\uparrow$ 53.25%)	0.991 $\pm$ 0.010
Lymphoma	0.2	0.275 $\pm$ 0.07($\uparrow$ 262%)	0.031 $\pm$ 0.003($\uparrow$ 3125.62%)	0.03 $\pm$ 0.004($\uparrow$ 3175.78%)	0.036 $\pm$ 0.004($\uparrow$ 2672.5%)	~	0.583 $\pm$ 0.102($\uparrow$ 70.58%)	0.994 $\pm$ 0.008
Lymphoma	0.3	0.245 $\pm$ 0.073($\uparrow$ 303.56%)	0.031 $\pm$ 0.005($\uparrow$ 3056.63%)	0.029 $\pm$ 0.001($\uparrow$ 3290.96%)	0.032 $\pm$ 0.003($\uparrow$ 2962.17%)	~	0.592 $\pm$ 0.068($\uparrow$ 66.71%)	0.987 $\pm$ 0.017
Lymphoma	0.4	0.286 $\pm$ 0.046($\uparrow$ 245.38%)	0.031 $\pm$ 0.002($\uparrow$ 3142.35%)	0.033 $\pm$ 0.003($\uparrow$ 2907.25%)	0.031 $\pm$ 0.003($\uparrow$ 3126.98%)	~	0.699 $\pm$ 0.083($\uparrow$ 41.48%)	0.989 $\pm$ 0.024
DLBCL	0.2	0.572 $\pm$ 0.07($\uparrow$ 62.85%)	0.041 $\pm$ 0.007($\uparrow$ 2151.75%)	0.291 $\pm$ 0.034($\uparrow$ 219.92%)	0.425 $\pm$ 0.068($\uparrow$ 118.88%)	~	0.519 $\pm$ 0.123($\uparrow$ 79.27%)	0.931 $\pm$ 0.086
DLBCL	0.3	0.469 $\pm$ 0.079($\uparrow$ 109.23%)	0.045 $\pm$ 0.008($\uparrow$ 2055.61%)	0.301 $\pm$ 0.007($\uparrow$ 225.94%)	0.358 $\pm$ 0.039($\uparrow$ 173.75%)	~	0.621 $\pm$ 0.04($\uparrow$ 57.78%)	0.980 $\pm$ 0.025
DLBCL	0.4	0.449 $\pm$ 0.1($\uparrow$ 115.22%)	0.042 $\pm$ 0.011($\uparrow$ 2224.35%)	0.314 $\pm$ 0.036($\uparrow$ 207.54%)	0.368 $\pm$ 0.058($\uparrow$ 162.39%)	~	0.487 $\pm$ 0.08($\uparrow$ 98.44%)	0.967 $\pm$ 0.042
Brain1	0.2	0.419 $\pm$ 0.085($\uparrow$ 127.22%)	0.038 $\pm$ 0.004($\uparrow$ 2416.63%)	0.302 $\pm$ 0.031($\uparrow$ 214.47%)	0.385 $\pm$ 0.066($\uparrow$ 146.95%)	~	0.414 $\pm$ 0.109($\uparrow$ 129.87%)	0.951 $\pm$ 0.082
Brain1	0.3	0.431 $\pm$ 0.095($\uparrow$ 102.53%)	0.037 $\pm$ 0.005($\uparrow$ 2243.63%)	0.296 $\pm$ 0.023($\uparrow$ 195.26%)	0.287 $\pm$ 0.05($\uparrow$ 204.29%)	~	0.442 $\pm$ 0.158($\uparrow$ 97.29%)	0.873 $\pm$ 0.240
Brain1	0.4	0.477 $\pm$ 0.108($\uparrow$ 99.18%)	0.033 $\pm$ 0.003($\uparrow$ 2807.23%)	0.3 $\pm$ 0.078($\uparrow$ 216.66%)	0.353 $\pm$ 0.05($\uparrow$ 169.13%)	~	0.596 $\pm$ 0.097($\uparrow$ 59.47%)	0.951 $\pm$ 0.098
Drivface	0.2	0.353 $\pm$ 0.105($\uparrow$ 182.89%)	0.037 $\pm$ 0.003($\uparrow$ 2598.27%)	0.038 $\pm$ 0.006($\uparrow$ 2539.27%)	0.051 $\pm$ 0.007($\uparrow$ 1844.71%)	~	0.504 $\pm$ 0.017($\uparrow$ 98.31%)	0.999 $\pm$ 0.001
Drivface	0.3	0.238 $\pm$ 0.026($\uparrow$ 318.77%)	0.041 $\pm$ 0.006($\uparrow$ 2337.53%)	0.037 $\pm$ 0.007($\uparrow$ 2619.66%)	0.054 $\pm$ 0.007($\uparrow$ 1763.47%)	~	0.551 $\pm$ 0.136($\uparrow$ 81.11%)	0.999 $\pm$ 0.004
Drivface	0.4	0.326 $\pm$ 0.059($\uparrow$ 198.57%)	0.043 $\pm$ 0.007($\uparrow$ 2180.29%)	0.035 $\pm$ 0.003($\uparrow$ 2673.13%)	0.055 $\pm$ 0.005($\uparrow$ 1662%)	~	0.467 $\pm$ 0.14($\uparrow$ 108.1%)	0.972 $\pm$ 0.071
Prostate	0.2	0.503 $\pm$ 0.062($\uparrow$ 92.04%)	0.039 $\pm$ 0.007($\uparrow$ 2356.92%)	0.32 $\pm$ 0.028($\uparrow$ 202.31%)	0.436 $\pm$ 0.062($\uparrow$ 121.72%)	~	0.537 $\pm$ 0.086($\uparrow$ 79.95%)	0.966 $\pm$ 0.071
Prostate	0.3	0.43 $\pm$ 0.058($\uparrow$ 126.08%)	0.039 $\pm$ 0.006($\uparrow$ 2390.86%)	0.286 $\pm$ 0.059($\uparrow$ 239.64%)	0.366 $\pm$ 0.045($\uparrow$ 165.98%)	~	0.481 $\pm$ 0.052($\uparrow$ 102.32%)	0.973 $\pm$ 0.057
Prostate	0.4	0.473 $\pm$ 0.099($\uparrow$ 94.31%)	0.04 $\pm$ 0.009($\uparrow$ 2175.77%)	0.254 $\pm$ 0.029($\uparrow$ 262.39%)	0.393 $\pm$ 0.025($\uparrow$ 133.72%)	~	0.491 $\pm$ 0.059($\uparrow$ 87.4%)	0.919 $\pm$ 0.204
GLI_85	0.2	0.516 $\pm$ 0.084($\uparrow$ 79.03%)	0.043 $\pm$ 0.002($\uparrow$ 2038.4%)	0.503 $\pm$ 0.037($\uparrow$ 83.41%)	---	~	0.167 $\pm$ 0.123($\uparrow$ 454.45%)	0.923 $\pm$ 0.127
GLI_85	0.3	0.498 $\pm$ 0.034($\uparrow$ 91.65%)	0.046 $\pm$ 0.004($\uparrow$ 1989.54%)	0.48 $\pm$ 0.045($\uparrow$ 98.5%)	---	~	0.324 $\pm$ 0.184($\uparrow$ 194.12%)	0.953 $\pm$ 0.077
GLI_85	0.4	0.502 $\pm$ 0.089($\uparrow$ 97.18%)	0.046 $\pm$ 0.002($\uparrow$ 2054.1%)	0.526 $\pm$ 0.043($\uparrow$ 88.09%)	---	~	0.431 $\pm$ 0.135($\uparrow$ 129.71%)	0.989 $\pm$ 0.019
win/tie/loss	0.2	10/0/0	10/0/0	10/0/0	10/0/0	-/-/-	10/0/0	-/-/-
	0.3	10/0/0	10/0/0	10/0/0	10/0/0	-/-/-	10/0/0	-/-/-
	0.4	10/0/0	10/0/0	10/0/0	10/0/0	-/-/-	10/0/0	-/-/-
avg-rank	0.2	2.5	5.7	4.9	4.3	-	2.6	1
	0.3	2.8	5.40	5.1	4.4	-	2.3	1
	0.4	2.7	5.7	4.7	4.4	-	2.5	1
p-value	0.2	1	0	0	0.0012	-	0.8374	-
	0.3	0.4717	0	0	0.0007	-	1	-
	0.4	0.6325	0	0.0001	0.0007	-	1	-

The symbol ‘-’ denotes the experimental results cannot be obtained in 24 h while the symbol ‘~’ represents that the sparsity in BAUC-OF is the same with the sparsity obtained by LMAO-PU. The percent in each parentheses denotes the improvement in sparsity using LMAO-PU, compared with the baseline.

4.2. Comparison algorithms and parameter settings

The performance of LMAO-PU is compared with six representative algorithms for PU learning, which are CL [28], uPU [14], PULD [32], PNU [33], BAUC-OF [44] and MOEA-PUL [17]. Among them, CL is a biased-based PU learning approach designed to tackle the issue of label noise when all U samples are treated as negative samples, which has shown its competitiveness compared with the sample-selection based approaches. uPU and PULD are the two representative class prior-based PU learning algorithms. Specifically, uPU is designed to obtain the PU classifier with high quality by developing different loss functions. In PULD, each U sample is treated as positive and negative sample simultaneously and the true label of the sample is identified by a margin-based label disambiguation strategy. PNU is a semi-supervised learning algorithm, which is regarded as a general algorithm and used for PU learning. MOEA-PUL is a recently suggested MOEA-based PU algorithm, where PU learning is firstly modeled as a multi-objective optimization problem and an MOEA is designed to tackled it, which has shown its superior performance than the state-of-the-arts. BAUC-OF is a recently suggested PU learning algorithm, which is closely related to our work. In BAUC-OF, the feature selection technique is employed to exclude the corrupted and noisy features, which can be used to

alleviate the issue of the curse of dimensionality and applied for sparse PU learning.

In our experiments, for the comparison algorithms, we employ the recommended parameters in their original papers [14,17,28,32,33,44] to get the best performance of the employed comparison algorithms. In LMAO-PU, the population size is set as 100 and the maximum number of iterations is fixed as 200, the number of iterations, denoted as *gen*, in the PU model optimization and sparse model optimization stages is set as 20, the size of neighbour of each unlabeled samples *k* is set to 10, the crossover probability and the mutation probability are respectively set to 0.7 and 0.4. To reduce the impact of randomness, the five-fold cross validation is employed on each dataset independently five times. We report the results of average sparsity, accuracy and AUC (area under ROC curve) of different algorithms (including the standard deviation), aiming to comprehensively evaluate the performance of sparse PU classifiers. In addition, we adopt t-test with a significance level of 0.05 and Friedman test with Holm post-hoc analysis to make the significant difference analysis statistically compared LMAO-PU with the other algorithms. All the experimental results are obtained on a PC with an Intel Core i5-9500 3.00 GHz CPU and Microsoft Windows 10 operating system.

Table 4

Average accuracy of LMAO-PU vs. CL, uPU, PNU, PULD, BAUC-OF and MOEA-PUL algorithms on the 10 datasets.

Dataset	$\mu\%$	CL [28]	uPU [14]	PNU [33]	PULD [32]	BAUC-OF [44]	MOEA-PUL [17]	LMAO-PU
LSVT	0.2	0.777 $\pm$ 0.074($\uparrow$ 16.34%)	0.692 $\pm$ 0.089($\uparrow$ 30.64%)	0.77 $\pm$ 0.103($\uparrow$ 17.43%)	0.869 $\pm$ 0.021($\uparrow$ 4%)	0.739 $\pm$ 0.132($\uparrow$ 22.37%)	0.682 $\pm$ 0.049($\uparrow$ 32.46%)	0.904 $\pm$ 0.054
LSVT	0.3	0.777 $\pm$ 0.083($\uparrow$ 11.2%)	0.732 $\pm$ 0.118($\uparrow$ 18.03%)	0.81 $\pm$ 0.076($\uparrow$ 6.69%)	0.846 $\pm$ 0.061($\uparrow$ 2.11%)	0.733 $\pm$ 0.095($\uparrow$ 17.86%)	0.738 $\pm$ 0.068($\uparrow$ 17.1%)	0.864 $\pm$ 0.095
LSVT	0.4	0.746 $\pm$ 0.059($\uparrow$ 14.71%)	0.732 $\pm$ 0.168($\uparrow$ 16.94%)	0.809 $\pm$ 0.118($\uparrow$ 5.78%)	0.785 $\pm$ 0.058($\uparrow$ 9.1%)	0.777 $\pm$ 0.088($\uparrow$ 10.15%)	0.65 $\pm$ 0.074($\uparrow$ 31.66%)	0.856 $\pm$ 0.063
Madelon	0.2	0.547 $\pm$ 0.011($\uparrow$ 7.26%)	0.503 $\pm$ 0.021($\uparrow$ 16.66%)	0.527 $\pm$ 0.018($\uparrow$ 11.34%)	0.498 $\pm$ 0.01($\uparrow$ 17.97%)	0.598 $\pm$ 0.021($\downarrow$ -1.9%)	0.566 $\pm$ 0.023($\uparrow$ 3.7%)	0.587 $\pm$ 0.042
Madelon	0.3	0.537 $\pm$ 0.011($\uparrow$ 12.66%)	0.505 $\pm$ 0.017($\uparrow$ 19.89%)	0.535 $\pm$ 0.017($\uparrow$ 13.17%)	0.497 $\pm$ 0.011($\uparrow$ 21.75%)	0.591 $\pm$ 0.015($\uparrow$ 2.44%)	0.573 $\pm$ 0.015($\uparrow$ 5.57%)	0.605 $\pm$ 0.037
Madelon	0.4	0.522 $\pm$ 0.012($\uparrow$ 5.5%)	0.495 $\pm$ 0.02($\uparrow$ 11.39%)	0.529 $\pm$ 0.027($\uparrow$ 4.14%)	0.502 $\pm$ 0.019($\uparrow$ 9.72%)	0.599 $\pm$ 0.015($\downarrow$ -8.05%)	0.555 $\pm$ 0.029($\downarrow$ -0.76%)	0.551 $\pm$ 0.047
Gse72526	0.2	0.923 $\pm$ 0.054($\uparrow$ 2.02%)	0.725 $\pm$ 0.142($\uparrow$ 29.89%)	0.724 $\pm$ 0.125($\uparrow$ 30%)	0.862 $\pm$ 0.126($\uparrow$ 9.3%)	0.921 $\pm$ 0.111($\uparrow$ 2.3%)	0.769 $\pm$ 0.073($\uparrow$ 22.42%)	0.942 $\pm$ 0.069
Gse72526	0.3	0.908 $\pm$ 0.1($\uparrow$ 5.59%)	0.7 $\pm$ 0.119($\uparrow$ 36.9%)	0.868 $\pm$ 0.127($\uparrow$ 10.41%)	0.785 $\pm$ 0.138($\uparrow$ 22.14%)	0.967 $\pm$ 0.042($\downarrow$ -0.93%)	0.773 $\pm$ 0.183($\uparrow$ 23.96%)	0.958 $\pm$ 0.044
Gse72526	0.4	0.877 $\pm$ 0.14($\uparrow$ 5.5%)	0.717 $\pm$ 0.105($\uparrow$ 29.07%)	0.886 $\pm$ 0.042($\uparrow$ 4.41%)	0.831 $\pm$ 0.084($\uparrow$ 11.34%)	0.865 $\pm$ 0.122($\uparrow$ 6.93%)	0.723 $\pm$ 0.167($\uparrow$ 27.93%)	0.925 $\pm$ 0.100
Colon	0.2	0.707 $\pm$ 0.034($\uparrow$ 31.94%)	0.65 $\pm$ 0.123($\uparrow$ 43.59%)	0.629 $\pm$ 0.091($\uparrow$ 48.27%)	0.662 $\pm$ 0.129($\uparrow$ 41.09%)	0.659 $\pm$ 0.172($\uparrow$ 41.63%)	0.649 $\pm$ 0.179($\uparrow$ 43.87%)	0.933 $\pm$ 0.077
Colon	0.3	0.754 $\pm$ 0.064($\uparrow$ 19.43%)	0.667 $\pm$ 0.104($\uparrow$ 35%)	0.563 $\pm$ 0.102($\uparrow$ 59.91%)	0.585 $\pm$ 0.103($\uparrow$ 53.95%)	0.731 $\pm$ 0.131($\uparrow$ 23.05%)	0.629 $\pm$ 0.115($\uparrow$ 42.97%)	0.900 $\pm$ 0.066
Colon	0.4	0.723 $\pm$ 0.129($\uparrow$ 21.06%)	0.717 $\pm$ 0.09($\uparrow$ 22.09%)	0.599 $\pm$ 0.105($\uparrow$ 46.15%)	0.646 $\pm$ 0.117($\uparrow$ 35.42%)	0.719 $\pm$ 0.155($\uparrow$ 21.75%)	0.632 $\pm$ 0.135($\uparrow$ 38.44%)	0.875 $\pm$ 0.090
Lymphoma	0.2	0.91 $\pm$ 0.042($\uparrow$ 0.06%)	0.753 $\pm$ 0.093($\uparrow$ 20.98%)	0.604 $\pm$ 0.142($\uparrow$ 50.7%)	0.85 $\pm$ 0.05($\uparrow$ 7.12%)	0.865 $\pm$ 0.077($\uparrow$ 5.23%)	0.876 $\pm$ 0.057($\uparrow$ 3.92%)	0.911 $\pm$ 0.066
Lymphoma	0.3	0.9 $\pm$ 0.1($\uparrow$ 4.68%)	0.779 $\pm$ 0.099($\uparrow$ 20.95%)	0.594 $\pm$ 0.164($\uparrow$ 58.69%)	0.84 $\pm$ 0.065($\uparrow$ 12.16%)	0.856 $\pm$ 0.104($\uparrow$ 10.09%)	0.886 $\pm$ 0.065($\uparrow$ 6.29%)	0.942 $\pm$ 0.063
Lymphoma	0.4	0.91 $\pm$ 0.055($\downarrow$ -0.52%)	0.789 $\pm$ 0.07($\uparrow$ 14.67%)	0.634 $\pm$ 0.126($\uparrow$ 42.74%)	0.82 $\pm$ 0.13($\uparrow$ 10.4%)	0.859 $\pm$ 0.072($\uparrow$ 5.34%)	0.834 $\pm$ 0.074($\uparrow$ 8.52%)	0.905 $\pm$ 0.065
DLBCL	0.2	0.975 $\pm$ 0.056($\uparrow$ 0.51%)	0.68 $\pm$ 0.112($\uparrow$ 44.12%)	0.507 $\pm$ 0.185($\uparrow$ 93.1%)	0.9 $\pm$ 0.071($\uparrow$ 8.89%)	0.753 $\pm$ 0.123($\uparrow$ 30.23%)	0.895 $\pm$ 0.111($\uparrow$ 9.5%)	0.980 $\pm$ 0.032
DLBCL	0.3	0.9 $\pm$ 0.035($\uparrow$ 3.68%)	0.653 $\pm$ 0.112($\uparrow$ 42.86%)	0.544 $\pm$ 0.074($\uparrow$ 71.52%)	0.813 $\pm$ 0.044($\uparrow$ 14.87%)	0.768 $\pm$ 0.132($\uparrow$ 21.61%)	0.821 $\pm$ 0.189($\uparrow$ 13.71%)	0.933 $\pm$ 0.070
DLBCL	0.4	0.825 $\pm$ 0.128($\uparrow$ 16.31%)	0.713 $\pm$ 0.104($\uparrow$ 34.58%)	0.56 $\pm$ 0.191($\uparrow$ 71.43%)	0.763 $\pm$ 0.103($\uparrow$ 25.9%)	0.771 $\pm$ 0.101($\uparrow$ 24.59%)	0.83 $\pm$ 0.077($\uparrow$ 15.66%)	0.960 $\pm$ 0.064
Brain1	0.2	0.878 $\pm$ 0.025($\uparrow$ 7.59%)	0.694 $\pm$ 0.118($\uparrow$ 36%)	0.656 $\pm$ 0.12($\uparrow$ 44.07%)	0.911 $\pm$ 0.03($\uparrow$ 3.66%)	0.822 $\pm$ 0.107($\uparrow$ 14.86%)	0.8 $\pm$ 0.101($\uparrow$ 18.06%)	0.944 $\pm$ 0.059
Brain1	0.3	0.856 $\pm$ 0.05($\uparrow$ 9.73%)	0.7 $\pm$ 0.115($\uparrow$ 34.13%)	0.6 $\pm$ 0.133($\uparrow$ 56.48%)	0.822 $\pm$ 0.091($\uparrow$ 14.19%)	0.794 $\pm$ 0.102($\uparrow$ 18.18%)	0.7 $\pm$ 0.15($\uparrow$ 34.13%)	0.939 $\pm$ 0.041
Brain1	0.4	0.767 $\pm$ 0.091($\uparrow$ 15.95%)	0.717 $\pm$ 0.089($\uparrow$ 24.03%)	0.633 $\pm$ 0.139($\uparrow$ 40.35%)	0.756 $\pm$ 0.108($\uparrow$ 17.65%)	0.794 $\pm$ 0.079($\uparrow$ 11.89%)	0.767 $\pm$ 0.061($\uparrow$ 15.94%)	0.889 $\pm$ 0.101
Drivface	0.2	0.901 $\pm$ 0.04($\uparrow$ 6.86%)	0.982 $\pm$ 0.019($\downarrow$ -1.95%)	0.967 $\pm$ 0.024($\downarrow$ -0.43%)	0.828 $\pm$ 0.036($\uparrow$ 16.28%)	0.66 $\pm$ 0.032($\uparrow$ 45.91%)	0.838 $\pm$ 0.042($\uparrow$ 14.89%)	0.963 $\pm$ 0.018
Drivface	0.3	0.835 $\pm$ 0.041($\uparrow$ 14.92%)	0.957 $\pm$ 0.023($\uparrow$ 0.18%)	0.962 $\pm$ 0.018($\downarrow$ -0.26%)	0.786 $\pm$ 0.031($\uparrow$ 22.01%)	0.678 $\pm$ 0.059($\uparrow$ 41.37%)	0.776 $\pm$ 0.065($\uparrow$ 23.68%)	0.959 $\pm$ 0.021
Drivface	0.4	0.702 $\pm$ 0.051($\uparrow$ 37.91%)	0.977 $\pm$ 0.015($\downarrow$ -0.89%)	0.96 $\pm$ 0.016($\uparrow$ 0.92%)	0.79 $\pm$ 0.045($\uparrow$ 22.69%)	0.606 $\pm$ 0.046($\uparrow$ 59.83%)	0.801 $\pm$ 0.054($\uparrow$ 20.96%)	0.969 $\pm$ 0.018
Prostate	0.2	0.857 $\pm$ 0.048($\uparrow$ 4.39%)	0.73 $\pm$ 0.136($\uparrow$ 22.6%)	0.698 $\pm$ 0.139($\uparrow$ 28.21%)	0.781 $\pm$ 0.054($\uparrow$ 14.6%)	0.921 $\pm$ 0.057($\downarrow$ -2.82%)	0.812 $\pm$ 0.068($\uparrow$ 10.17%)	0.895 $\pm$ 0.069
Prostate	0.3	0.819 $\pm$ 0.085($\uparrow$ 4.37%)	0.74 $\pm$ 0.117($\uparrow$ 15.54%)	0.616 $\pm$ 0.13($\uparrow$ 38.86%)	0.705 $\pm$ 0.109($\uparrow$ 21.32%)	0.871 $\pm$ 0.06($\downarrow$ -1.88%)	0.718 $\pm$ 0.178($\uparrow$ 19.14%)	0.855 $\pm$ 0.069
Prostate	0.4	0.752 $\pm$ 0.179($\uparrow$ 14.3%)	0.76 $\pm$ 0.107($\uparrow$ 13.16%)	0.629 $\pm$ 0.125($\uparrow$ 36.82%)	0.714 $\pm$ 0.121($\uparrow$ 20.4%)	0.868 $\pm$ 0.045($\downarrow$ -0.98%)	0.753 $\pm$ 0.138($\uparrow$ 14.16%)	0.860 $\pm$ 0.074
GLI_85	0.2	0.788 $\pm$ 0.067($\uparrow$ 14.16%)	0.671 $\pm$ 0.093($\uparrow$ 34.21%)	0.612 $\pm$ 0.089($\uparrow$ 47.12%)	---	0.765 $\pm$ 0.093($\uparrow$ 17.69%)	0.753 $\pm$ 0.174($\uparrow$ 19.53%)	0.900 $\pm$ 0.079
GLI_85	0.3	0.894 $\pm$ 0.087($\uparrow$ 0.01%)	0.659 $\pm$ 0.095($\uparrow$ 35.71%)	0.553 $\pm$ 0.169($\uparrow$ 61.7%)	---	0.787 $\pm$ 0.098($\uparrow$ 13.67%)	0.847 $\pm$ 0.129($\uparrow$ 5.56%)	0.894 $\pm$ 0.077
GLI_85	0.4	0.823 $\pm$ 0.156($\uparrow$ 1.44%)	0.729 $\pm$ 0.084($\uparrow$ 14.52%)	0.529 $\pm$ 0.208($\uparrow$ 57.78%)	---	0.822 $\pm$ 0.11($\uparrow$ 1.68%)	0.706 $\pm$ 0.161($\uparrow$ 18.33%)	0.835 $\pm$ 0.091
win/tie/loss	0.2	6/4/0	9/1/0	9/1/0	7/3/0	6/4/0	8/2/0	-/-/-
	0.3	6/4/0	9/1/0	8/2/0	9/1/0	7/3/0	9/1/0	-/-/-
	0.4	6/4/0	9/1/0	6/4/0	9/1/0	5/4/1	9/1/0	-/-/-
avg-rank	0.2	2.7	5.3	5.9	4.4	4.6	3.7	1.4
	0.3	2.8	5.3	5.4	5	4.5	3.7	1.3
	0.4	3.6	5	5.2	5.2	4.3	3.2	1.5
p-value	0.2	1	0.0011	0.0001	0.0399	0.0194	0.3628	-
	0.3	1	0.0007	0.0005	0.0027	0.0194	0.2726	-
	0.4	0.6243	0.0061	0.0027	0.0027	0.0788	1	-

The symbol ‘-’ denotes the experimental results cannot be obtained in 24 h. The percent in each parentheses denotes the improvement in accuracy using LMAO-PU, compared with the baseline.

4.3. The comparison results between LMAO-PU and baselines

Table 3 reports the average sparsity and standard deviations of the PU classifiers obtained by LMAO-PU and six baselines, where the maximum sparsity on each dataset is marked with grey. “win/tie/loss” is the results of t-test, which is used to denote the result of ours that are significantly better, statistically similar and significantly worse than that of those obtained by the other comparison algorithm, respectively. “avg-rank” is the results of Friedman test used to compare the performance of the proposed LMAO-PU among all the algorithms. For CL, uPU, PULD, PNU and MOEA-PUL, the percent of zero elements in the PU classifier is regarded as the sparsity of the PU classifier.⁵ For BAUC-OF, the sparsity is a predefined parameter in the original paper. For fair comparison, in all the experiments, the sparsity in BAUC-OF is set as the same with the sparsity obtained by LMAO-PU. From this table, LMAO-PU shows the significant superiority to the other baselines on all datasets. For example, on *GLI_85* dataset, which has 22,283 features, the other comparison algorithms only achieve

the average sparsity around 0.5 while the proposed LMAO-PU can obtain the highest sparsity value of 0.98, which shows our LMAO-PU can achieve the PU classifiers with higher sparsity. In addition, the significance statistics of t-test reveal that LMAO-PU is always significantly better than all the other comparison algorithm and never tie or lose for once. The reason is mainly attributed to the fact that different from the existing PU algorithms only focusing on the accuracy of PU classifiers, in LMAO-PU, a novel evolutionary based alternating optimization framework is proposed for high-dimensional PU learning. In the suggested framework, the accuracy and sparsity of PU model are optimized alternately, which can ensure the quality of PU classifier and obtain higher sparsity.

To further verify the classification performance of the obtained sparse PU classifiers, Table 4 gives the average accuracy of the achieved PU classifiers on all experimental datasets, where the best results are marked in grey. From this table, we can find that the proposed PU model optimization can obtain better classification accuracy than the others. Especially on the highest dimensional dataset *GLI_85*, the LMAO-PU still achieves the highest classification accuracy no matter when $\mu = 0.2, 0.3$ or 0.4 . The significance statistics also justify our superiority in terms of classification accuracy, that is the proposed LMAO-PU almost always wins or ties on all datasets, no lose for once.

⁵ Usually, the weight in PU classifier is smaller than 0.01, the corresponding weight is set to 0.

Table 5

Average AUC of LMAO-PU vs. CL, uPU, PNU, PULD, BAUC-OF and MOEA-PUL algorithms on the 10 datasets.

Dataset	$\mu\%$	CL [28]	uPU [14]	PNU [33]	PULD [32]	BAUC-OF [44]	MOEA-PUL [17]	LMAO-PU
LSVT	0.2	0.709 $\pm$ 0.286($\uparrow$ 37.5%)	0.894 $\pm$ 0.052($\uparrow$ 9.04%)	0.893 $\pm$ 0.062($\uparrow$ 9.1%)	0.909 $\pm$ 0.053($\uparrow$ 7.24%)	0.861 $\pm$ 0.131($\uparrow$ 13.13%)	0.786 $\pm$ 0.059($\uparrow$ 24.01%)	0.974 $\pm$ 0.017
LSVT	0.3	0.726 $\pm$ 0.29($\uparrow$ 20.63%)	0.881 $\pm$ 0.048($\downarrow$ -0.69%)	0.873 $\pm$ 0.087($\uparrow$ 0.25%)	0.909 $\pm$ 0.059($\downarrow$ -3.66%)	0.85 $\pm$ 0.087($\uparrow$ 2.93%)	0.779 $\pm$ 0.129($\uparrow$ 12.44%)	0.875 $\pm$ 0.073
LSVT	0.4	0.699 $\pm$ 0.278($\uparrow$ 26.69%)	0.834 $\pm$ 0.078($\uparrow$ 6.06%)	0.836 $\pm$ 0.144($\uparrow$ 5.85%)	0.843 $\pm$ 0.036($\uparrow$ 4.98%)	0.803 $\pm$ 0.062($\uparrow$ 10.23%)	0.69 $\pm$ 0.075($\uparrow$ 28.3%)	0.885 $\pm$ 0.047
Madelon	0.2	0.486 $\pm$ 0.209($\uparrow$ 28.98%)	0.613 $\pm$ 0.037($\uparrow$ 2.24%)	0.54 $\pm$ 0.03($\uparrow$ 15.9%)	0.516 $\pm$ 0.019($\uparrow$ 21.44%)	0.606 $\pm$ 0.02($\uparrow$ 3.38%)	0.581 $\pm$ 0.022($\uparrow$ 7.75%)	0.626 $\pm$ 0.009
Madelon	0.3	0.488 $\pm$ 0.209($\uparrow$ 30.83%)	0.606 $\pm$ 0.037($\uparrow$ 5.43%)	0.544 $\pm$ 0.02($\uparrow$ 17.32%)	0.502 $\pm$ 0.012($\uparrow$ 27.27%)	0.562 $\pm$ 0.028($\uparrow$ 13.54%)	0.577 $\pm$ 0.009($\uparrow$ 10.67%)	0.638 $\pm$ 0.134
Madelon	0.4	0.476 $\pm$ 0.204($\uparrow$ 21.33%)	0.600 $\pm$ 0.037($\downarrow$ -3.73%)	0.536 $\pm$ 0.032($\uparrow$ 7.72%)	0.507 $\pm$ 0.019($\uparrow$ 14.01%)	0.55 $\pm$ 0.056($\uparrow$ 4.97%)	0.565 $\pm$ 0.036($\uparrow$ 2.27%)	0.578 $\pm$ 0.027
Gse72526	0.2	0.764 $\pm$ 0.308($\uparrow$ 28.92%)	0.902 $\pm$ 0.032($\uparrow$ 9.23%)	0.969 $\pm$ 0.054($\uparrow$ 1.64%)	0.888 $\pm$ 0.088($\uparrow$ 10.96%)	0.918 $\pm$ 0.052($\uparrow$ 7.3%)	0.892 $\pm$ 0.135($\uparrow$ 10.48%)	0.985 $\pm$ 0.033
Gse72526	0.3	0.778 $\pm$ 0.312($\uparrow$ 26.68%)	0.878 $\pm$ 0.086($\uparrow$ 12.38%)	0.946 $\pm$ 0.074($\uparrow$ 4.19%)	0.862 $\pm$ 0.104($\uparrow$ 14.36%)	0.975 $\pm$ 0.02($\uparrow$ 1.15%)	0.865 $\pm$ 0.133($\uparrow$ 14.04%)	0.986 $\pm$ 0.044
Gse72526	0.4	0.800 $\pm$ 0.320($\uparrow$ 20.36%)	0.864 $\pm$ 0.059($\uparrow$ 11.43%)	0.947 $\pm$ 0.062($\uparrow$ 1.68%)	0.896 $\pm$ 0.062($\uparrow$ 7.49%)	0.903 $\pm$ 0.088($\uparrow$ 6.68%)	0.792 $\pm$ 0.219($\uparrow$ 21.51%)	0.963 $\pm$ 0.079
Colon	0.2	0.755 $\pm$ 0.308($\uparrow$ 22.81%)	0.89 $\pm$ 0.048($\uparrow$ 4.2%)	0.874 $\pm$ 0.088($\uparrow$ 6.08%)	0.767 $\pm$ 0.112($\uparrow$ 20.82%)	0.648 $\pm$ 0.171($\uparrow$ 42.97%)	0.736 $\pm$ 0.16($\uparrow$ 26%)	0.927 $\pm$ 0.094
Colon	0.3	0.753 $\pm$ 0.321($\uparrow$ 28.37%)	0.86 $\pm$ 0.047($\uparrow$ 12.39%)	0.787 $\pm$ 0.171($\uparrow$ 22.77%)	0.69 $\pm$ 0.123($\uparrow$ 39.92%)	0.678 $\pm$ 0.252($\uparrow$ 42.53%)	0.74 $\pm$ 0.132($\uparrow$ 30.59%)	0.966 $\pm$ 0.061
Colon	0.4	0.777 $\pm$ 0.326($\uparrow$ 25.43%)	0.855 $\pm$ 0.084($\uparrow$ 13.88%)	0.841 $\pm$ 0.14($\uparrow$ 15.78%)	0.67 $\pm$ 0.156($\uparrow$ 45.27%)	0.678 $\pm$ 0.087($\uparrow$ 43.63%)	0.714 $\pm$ 0.151($\uparrow$ 36.34%)	0.974 $\pm$ 0.062
Lymphoma	0.2	0.835 $\pm$ 0.357($\uparrow$ 15.41%)	0.895 $\pm$ 0.044($\uparrow$ 7.69%)	0.934 $\pm$ 0.047($\uparrow$ 3.11%)	0.871 $\pm$ 0.026($\uparrow$ 10.61%)	0.934 $\pm$ 0.053($\uparrow$ 3.12%)	0.987 $\pm$ 0.018($\downarrow$ -2.37%)	0.963 $\pm$ 0.036
Lymphoma	0.3	0.842 $\pm$ 0.356($\uparrow$ 16.24%)	0.897 $\pm$ 0.032($\uparrow$ 9.16%)	0.953 $\pm$ 0.036($\uparrow$ 2.71%)	0.857 $\pm$ 0.063($\uparrow$ 14.25%)	0.917 $\pm$ 0.044($\uparrow$ 6.79%)	0.947 $\pm$ 0.032($\uparrow$ 3.32%)	0.979 $\pm$ 0.044
Lymphoma	0.4	0.831 $\pm$ 0.348($\uparrow$ 16.97%)	0.851 $\pm$ 0.057($\uparrow$ 14.25%)	0.948 $\pm$ 0.041($\uparrow$ 2.48%)	0.798 $\pm$ 0.161($\uparrow$ 21.76%)	0.829 $\pm$ 0.081($\uparrow$ 17.23%)	0.954 $\pm$ 0.033($\uparrow$ 1.93%)	0.972 $\pm$ 0.037
DLBCL	0.2	0.857 $\pm$ 0.378($\uparrow$ 14.62%)	0.863 $\pm$ 0.088($\uparrow$ 13.86%)	0.895 $\pm$ 0.146($\uparrow$ 9.82%)	0.936 $\pm$ 0.053($\uparrow$ 4.96%)	0.895 $\pm$ 0.064($\uparrow$ 9.76%)	0.954 $\pm$ 0.065($\uparrow$ 2.98%)	0.982 $\pm$ 0.027
DLBCL	0.3	0.842 $\pm$ 0.356($\uparrow$ 15.67%)	0.888 $\pm$ 0.073($\uparrow$ 9.64%)	0.801 $\pm$ 0.083($\uparrow$ 21.59%)	0.895 $\pm$ 0.075($\uparrow$ 8.79%)	0.937 $\pm$ 0.025($\uparrow$ 3.93%)	0.91 $\pm$ 0.145($\uparrow$ 7.02%)	0.974 $\pm$ 0.04
DLBCL	0.4	0.814 $\pm$ 0.334($\uparrow$ 21.03%)	0.88 $\pm$ 0.068($\uparrow$ 11.99%)	0.838 $\pm$ 0.159($\uparrow$ 17.55%)	0.834 $\pm$ 0.085($\uparrow$ 18.19%)	0.874 $\pm$ 0.065($\uparrow$ 12.67%)	0.915 $\pm$ 0.114($\uparrow$ 7.69%)	0.985 $\pm$ 0.027
Brain1	0.2	0.805 $\pm$ 0.333($\uparrow$ 21.32%)	0.867 $\pm$ 0.059($\uparrow$ 12.66%)	0.902 $\pm$ 0.108($\uparrow$ 8.22%)	0.956 $\pm$ 0.063($\uparrow$ 2.19%)	0.795 $\pm$ 0.092($\uparrow$ 22.85%)	0.877 $\pm$ 0.136($\uparrow$ 11.37%)	0.976 $\pm$ 0.044
Brain1	0.3	0.802 $\pm$ 0.331($\uparrow$ 17.78%)	0.853 $\pm$ 0.036($\uparrow$ 10.82%)	0.882 $\pm$ 0.096($\uparrow$ 7.15%)	0.886 $\pm$ 0.088($\uparrow$ 6.64%)	0.847 $\pm$ 0.108($\uparrow$ 11.63%)	0.802 $\pm$ 0.116($\uparrow$ 17.79%)	0.945 $\pm$ 0.083
Brain1	0.4	0.789 $\pm$ 0.324($\uparrow$ 19.81%)	0.873 $\pm$ 0.044($\uparrow$ 8.32%)	0.851 $\pm$ 0.115($\uparrow$ 11.15%)	0.764 $\pm$ 0.054($\uparrow$ 23.82%)	0.767 $\pm$ 0.127($\uparrow$ 23.37%)	0.767 $\pm$ 0.177($\uparrow$ 23.27%)	0.946 $\pm$ 0.063
Drivface	0.2	0.801 $\pm$ 0.323($\uparrow$ 8.38%)	0.892 $\pm$ 0.044($\downarrow$ -2.75%)	0.963 $\pm$ 0.047($\downarrow$ -9.9%)	0.856 $\pm$ 0.115($\uparrow$ 1.33%)	0.823 $\pm$ 0.074($\uparrow$ 5.39%)	0.789 $\pm$ 0.13($\uparrow$ 10.04%)	0.868 $\pm$ 0.12
Drivface	0.3	0.838 $\pm$ 0.363($\uparrow$ 2.57%)	0.875 $\pm$ 0.051($\downarrow$ -1.84%)	0.945 $\pm$ 0.046($\downarrow$ -9.04%)	0.785 $\pm$ 0.078($\uparrow$ 9.46%)	0.887 $\pm$ 0.025($\downarrow$ -3.13%)	0.77 $\pm$ 0.098($\uparrow$ 11.63%)	0.859 $\pm$ 0.158
Drivface	0.4	0.823 $\pm$ 0.349($\uparrow$ 9.44%)	0.896 $\pm$ 0.041($\uparrow$ 0.59%)	0.971 $\pm$ 0.023($\downarrow$ -7.18%)	0.671 $\pm$ 0.117($\uparrow$ 34.23%)	0.875 $\pm$ 0.01($\uparrow$ 3.05%)	0.794 $\pm$ 0.065($\uparrow$ 13.53%)	0.901 $\pm$ 0.121
Prostate	0.2	0.809 $\pm$ 0.342($\uparrow$ 13.35%)	0.883 $\pm$ 0.041($\uparrow$ 3.9%)	0.839 $\pm$ 0.101($\uparrow$ 9.35%)	0.887 $\pm$ 0.04($\uparrow$ 3.39%)	0.905 $\pm$ 0.087($\uparrow$ 1.33%)	0.91 $\pm$ 0.085($\uparrow$ 0.83%)	0.917 $\pm$ 0.047
Prostate	0.3	0.825 $\pm$ 0.347($\uparrow$ 15.71%)	0.879 $\pm$ 0.061($\uparrow$ 8.56%)	0.793 $\pm$ 0.116($\uparrow$ 20.34%)	0.778 $\pm$ 0.061($\uparrow$ 22.65%)	0.858 $\pm$ 0.126($\uparrow$ 11.19%)	0.83 $\pm$ 0.15($\uparrow$ 15.02%)	0.954 $\pm$ 0.042
Prostate	0.4	0.771 $\pm$ 0.324($\uparrow$ 23.31%)	0.854 $\pm$ 0.053($\uparrow$ 11.44%)	0.717 $\pm$ 0.082($\uparrow$ 32.75%)	0.773 $\pm$ 0.149($\uparrow$ 23.1%)	0.856 $\pm$ 0.103($\uparrow$ 11.1%)	0.847 $\pm$ 0.087($\uparrow$ 12.27%)	0.951 $\pm$ 0.041
GLI_85	0.2	0.744 $\pm$ 0.306($\uparrow$ 32.51%)	0.869 $\pm$ 0.038($\uparrow$ 13.37%)	0.942 $\pm$ 0.109($\uparrow$ 4.6%)	---	0.872 $\pm$ 0.056($\uparrow$ 13.01%)	0.81 $\pm$ 0.11($\uparrow$ 21.66%)	0.986 $\pm$ 0.020
GLI_85	0.3	0.807 $\pm$ 0.335($\uparrow$ 16.53%)	0.867 $\pm$ 0.056($\uparrow$ 8.52%)	0.862 $\pm$ 0.164($\uparrow$ 9.18%)	---	0.868 $\pm$ 0.1($\uparrow$ 8.33%)	0.905 $\pm$ 0.074($\uparrow$ 3.91%)	0.941 $\pm$ 0.081
GLI_85	0.4	0.793 $\pm$ 0.321($\uparrow$ 6.85%)	0.862 $\pm$ 0.048($\downarrow$ -1.75%)	0.824 $\pm$ 0.232($\uparrow$ 2.85%)	---	0.835 $\pm$ 0.092($\uparrow$ 1.49%)	0.725 $\pm$ 0.135($\uparrow$ 16.79%)	0.847 $\pm$ 0.099
win/tie/loss	0.2	6/4/0	7/3/0	7/2/1	8/2/0	8/2/0	6/3/1	-/-/-
	0.3	6/4/0	7/3/0	8/1/1	9/1/0	8/2/0	9/1/0	-/-/-
	0.4	8/2/0	7/2/1	6/3/1	10/0/0	8/2/0	9/1/0	-/-/-
avg-rank	0.2	6.6	3.9	3.5	3.8	4.3	4.5	1.4
	0.3	6.1	3.4	4	4.5	4.6	3.8	1.6
	0.4	5.6	2.9	3.7	5.2	4.8	4.4	1.4
p-value	0.2	0.0000	0.2029	0.6243	0.2726	0.0564	0.028	-
	0.3	0.0001	1	0.2726	0.0564	0.0399	0.4782	-
	0.4	0.0003	1	0.3628	0.0018	0.0091	0.0399	-

The symbol '-' denotes the experimental results cannot be obtained in 24 h. The percent in each parentheses denotes the improvement in AUC using LMAO-PU, compared with the baseline.

Compared with BAUC-PU, which has the same sparsity with LMAO-PU, the proposed LMAO-PU still achieves the highest classification accuracy. The reason lies in the fact that in the stage of PU model optimization, a loss matrix, which utilizes the information of neighbor samples of each U sample, is proposed. The designed loss matrix can be used to calibrate the classification performance of PU classifiers, by which we can obtain the PU classifier with high classification accuracy.

Since the true labels of testing data are known in our experiments, thus, we employ the metric of AUC to compare the performance of different algorithms and the results on all datasets are shown in Table 5. From the table, compared with other comparison algorithms, the proposed LMAO-PU achieves the highest AUC values on most of datasets. The significance statistics also justify our superiority in terms of AUC. All these results shown in Tables 3 to 5 have demonstrated that the proposed LMAO-PU could not only obtain the higher sparseness PU classifiers but also have better classification accuracy, which can be used to tackle the challenge of PU learning under the situation of high-dimensional PU data.

In addition, we have provided the Friedman test with Holm post-hoc analysis of six PU algorithms in terms of sparsity, accuracy and AUC. The experimental results are shown in the last two lines of Tables 3–5. From the significant analysis results, we can obtain the corresponding p -value of Friedman test, which means that at $\alpha = 0.05$, there is

evidence to reject the null hypothesis and the six PU algorithms are significantly different. The avg-ranks in each table indicate that the proposed LMAO-PU obtains the best average rank no matter measured by sparsity, accuracy or AUC. This fact has demonstrated the superiority of LMAO-PU, compared with other state-of-the-arts. We take Table 3 as an example, and give a deeper Post-hoc analysis on it. Compared with uPU, PNU, PULD and MOEA-PUL, the p -values are all below 0.05, which means LMAO-PU achieve better sparsity than these four algorithms, and the difference is statistically relevant. When compared with CL, the p -value is larger than 0.05. It shows that although the average rank of LMAO-PU is better than CL, their difference is not statistically relevant. All those results have further demonstrated our proposed method could obtain the sparse PU classifier with high quality in high-dimensional PU learning. The reason is mainly attributed to the suggested two-stage alternating optimization in LMAO-PU. Specifically, the first stage is used to obtain the optimal PU models where the second stage is used to perform the sparse optimization, which are together to find the sparse PU models.

4.4. Effectiveness of the proposed alternating optimization

The super performance of LMAO-PU is attributed to the designed alternating optimization approach, in which the PU model and sparse

Table 6

The comparison results between LMAO-PU and LMAO-PU-NoAO on the 10 datasets.

Dataset	μ %	LMAO-PU-NoAO (Sparsity)	LMAO-PU (Sparsity)	LMAO-PU-NoAO (Accuracy)	LMAO-PU (Accuracy)
<i>LSVT</i>	0.2	0.230 $\pm$ 0.218	0.942 $\pm$ 0.061	0.818 $\pm$ 0.032	0.904 $\pm$ 0.054
<i>LSVT</i>	0.3	0.034 $\pm$ 0.037	0.910 $\pm$ 0.137	0.810 $\pm$ 0.030	0.864 $\pm$ 0.095
<i>LSVT</i>	0.4	0.267 $\pm$ 0.047	0.930 $\pm$ 0.086	0.761 $\pm$ 0.065	0.856 $\pm$ 0.063
<i>Madelon</i>	0.2	0.066 $\pm$ 0.130	0.988 $\pm$ 0.014	0.572 $\pm$ 0.027	0.587 $\pm$ 0.042
<i>Madelon</i>	0.3	0.112 $\pm$ 0.236	0.961 $\pm$ 0.080	0.535 $\pm$ 0.015	0.605 $\pm$ 0.037
<i>Madelon</i>	0.4	0.537 $\pm$ 0.017	0.982 $\pm$ 0.020	0.541 $\pm$ 0.018	0.551 $\pm$ 0.047
<i>Gse72526</i>	0.2	0.283 $\pm$ 0.230	0.961 $\pm$ 0.093	0.903 $\pm$ 0.067	0.942 $\pm$ 0.069
<i>Gse72526</i>	0.3	0.320 $\pm$ 0.260	0.991 $\pm$ 0.007	0.837 $\pm$ 0.098	0.958 $\pm$ 0.044
<i>Gse72526</i>	0.4	0.347 $\pm$ 0.262	0.995 $\pm$ 0.003	0.786 $\pm$ 0.141	0.925 $\pm$ 0.100
<i>Colon</i>	0.2	0.367 $\pm$ 0.221	0.897 $\pm$ 0.108	0.824 $\pm$ 0.082	0.933 $\pm$ 0.077
<i>Colon</i>	0.3	0.440 $\pm$ 0.174	0.954 $\pm$ 0.092	0.777 $\pm$ 0.097	0.900 $\pm$ 0.066
<i>Colon</i>	0.4	0.349 $\pm$ 0.234	0.991 $\pm$ 0.010	0.692 $\pm$ 0.107	0.875 $\pm$ 0.090
<i>Lymphoma</i>	0.2	0.128 $\pm$ 0.209	0.994 $\pm$ 0.008	0.886 $\pm$ 0.042	0.911 $\pm$ 0.066
<i>Lymphoma</i>	0.3	0.257 $\pm$ 0.154	0.987 $\pm$ 0.017	0.864 $\pm$ 0.080	0.942 $\pm$ 0.063
<i>Lymphoma</i>	0.4	0.214 $\pm$ 0.271	0.989 $\pm$ 0.024	0.874 $\pm$ 0.080	0.905 $\pm$ 0.065
<i>DLBCL</i>	0.2	0.296 $\pm$ 0.210	0.931 $\pm$ 0.086	0.832 $\pm$ 0.055	0.980 $\pm$ 0.032
<i>DLBCL</i>	0.3	0.373 $\pm$ 0.199	0.980 $\pm$ 0.025	0.829 $\pm$ 0.155	0.933 $\pm$ 0.070
<i>DLBCL</i>	0.4	0.504 $\pm$ 0.009	0.967 $\pm$ 0.042	0.767 $\pm$ 0.115	0.960 $\pm$ 0.064
<i>Brain1</i>	0.2	0.130 $\pm$ 0.219	0.951 $\pm$ 0.082	0.800 $\pm$ 0.075	0.944 $\pm$ 0.059
<i>Brain1</i>	0.3	0.425 $\pm$ 0.200	0.873 $\pm$ 0.240	0.800 $\pm$ 0.030	0.939 $\pm$ 0.041
<i>Brain1</i>	0.4	0.354 $\pm$ 0.133	0.951 $\pm$ 0.098	0.767 $\pm$ 0.072	0.889 $\pm$ 0.101
<i>Drivface</i>	0.2	0.079 $\pm$ 0.070	0.999 $\pm$ 0.001	0.977 $\pm$ 0.015	0.963 $\pm$ 0.018
<i>Drivface</i>	0.3	0.106 $\pm$ 0.109	0.999 $\pm$ 0.004	0.965 $\pm$ 0.009	0.959 $\pm$ 0.021
<i>Drivface</i>	0.4	0.152 $\pm$ 0.109	0.972 $\pm$ 0.071	0.974 $\pm$ 0.021	0.969 $\pm$ 0.018
<i>Prostate</i>	0.2	0.192 $\pm$ 0.203	0.966 $\pm$ 0.071	0.814 $\pm$ 0.021	0.895 $\pm$ 0.069
<i>Prostate</i>	0.3	0.207 $\pm$ 0.186	0.973 $\pm$ 0.057	0.813 $\pm$ 0.056	0.855 $\pm$ 0.069
<i>Prostate</i>	0.4	0.258 $\pm$ 0.233	0.919 $\pm$ 0.204	0.744 $\pm$ 0.140	0.860 $\pm$ 0.074
<i>GLI_85</i>	0.2	0.289 $\pm$ 0.203	0.923 $\pm$ 0.127	0.776 $\pm$ 0.105	0.900 $\pm$ 0.079
<i>GLI_85</i>	0.3	0.330 $\pm$ 0.168	0.953 $\pm$ 0.077	0.800 $\pm$ 0.089	0.894 $\pm$ 0.077
<i>GLI_85</i>	0.4	0.504 $\pm$ 0.004	0.989 $\pm$ 0.019	0.741 $\pm$ 0.067	0.835 $\pm$ 0.091
win/tie/loss	0.2	10/0/0	-/-/-	6/4/0	-/-/-
	0.3	10/0/0	-/-/-	6/4/0	-/-/-
	0.4	10/0/0	-/-/-	7/3/0	-/-/-

model are alternately optimized to obtain the optimal sparse PU classifier. To validate the effectiveness of the proposed alternating optimization, we compare the proposed method with LMAO-PU-NoAO, which is the same algorithm as ours, and the only difference is the alternating optimization with two-stage is replaced by a one-stage optimization. Specifically, in the one-stage optimization of LMAO-PU-NoAO, two objectives, the sparsity and the accuracy, are employed to build the one-stage sparse PU model.

Table 6 reports the comparison results between the LMAO-PU and LMAO-PU-NoAO on 10 real datasets. The results shown in the table indicate that due to the designed alternating optimization method, the PU classifiers obtained by LMAO-PU illustrate the high accuracy and achieve much better sparsity than those obtained by LMAO-PU-NoAO. The statistical significance also indicates that on the *Sparsity*, LMAO-PU always wins LMAO-PU-NoAO, and never tie or lose. Meanwhile, on the *Accuracy*, LMAO-PU also shows its superiority, since it always wins or ties LMAO-PU-NoAO, and never lose. We note that in one-stage optimization model LMAO-PU-NoAO, the accuracy of the obtained PU classifiers is comparable to those of PU classifiers obtained by the two-stage alternating optimization model, but the sparsity is obviously worse than those of the two-stage alternating optimization model

LMAO-PU. The reason is attributed to the fact that although LMAO-PU-NoAO also adopts accuracy and sparsity as the two optimized objectives, it is difficult to achieve higher classification accuracy of PU classifiers due to the label uncertainty of U samples. Different from LMAO-PU-NoAO, in our proposed algorithm, we design an alternating optimization framework to achieve the PU classifiers with higher accuracy and sparsity. Specifically, the first stage of LMAO-PU is to optimize PU model, aiming to obtain the PU classifiers with high accuracy by tackling the label uncertainty of U samples, with which to perform the second stage of sparse optimization to ensure the better sparsity. The two stages are performed alternately to achieve the PU classifiers with better accuracy and sparsity.

4.5. Effectiveness of loss matrix

To implement the proposed alternating optimization, a loss matrix is suggested, which is used to evaluate the performance of PU classifiers on U samples. In the following, we further verify the effectiveness of the suggested loss matrix in LMAO-PU. Table 7 gives the results of LMAO-PU and its variant, i.e., LMAO-PU-NoLM, on all the experimental datasets in terms of *Sparsity* and *Accuracy*. To be specific, LMAO-PU-NoLM is the same as LMAO-PU, except that the suggested loss

Table 7

The comparison results between LMAO-PU and LMAO-PU-NoLM on the 10 datasets.

Dataset	μ %	LMAO-PU-NoLM (Sparsity)	LMAO-PU (Sparsity)	LMAO-PU-NoLM (Accuracy)	LMAO-PU (Accuracy)
LSVT	0.2	0.748 $\pm$ 0.322	0.942 $\pm$ 0.061	0.770 $\pm$ 0.077	0.904 $\pm$ 0.054
LSVT	0.3	0.919 $\pm$ 0.089	0.910 $\pm$ 0.137	0.794 $\pm$ 0.128	0.864 $\pm$ 0.095
LSVT	0.4	0.955 $\pm$ 0.085	0.930 $\pm$ 0.086	0.738 $\pm$ 0.079	0.856 $\pm$ 0.063
Madelon	0.2	0.900 $\pm$ 0.134	0.988 $\pm$ 0.014	0.581 $\pm$ 0.056	0.587 $\pm$ 0.042
Madelon	0.3	0.842 $\pm$ 0.342	0.961 $\pm$ 0.08	0.506 $\pm$ 0.026	0.605 $\pm$ 0.037
Madelon	0.4	0.997 $\pm$ 0.002	0.982 $\pm$ 0.02	0.501 $\pm$ 0.037	0.551 $\pm$ 0.047
Gse72526	0.2	0.972 $\pm$ 0.040	0.961 $\pm$ 0.093	0.936 $\pm$ 0.065	0.942 $\pm$ 0.069
Gse72526	0.3	0.993 $\pm$ 0.009	0.991 $\pm$ 0.007	0.869 $\pm$ 0.073	0.958 $\pm$ 0.044
Gse72526	0.4	0.879 $\pm$ 0.160	0.995 $\pm$ 0.003	0.886 $\pm$ 0.093	0.925 $\pm$ 0.100
Colon	0.2	0.973 $\pm$ 0.035	0.897 $\pm$ 0.108	0.808 $\pm$ 0.134	0.933 $\pm$ 0.077
Colon	0.3	0.900 $\pm$ 0.179	0.954 $\pm$ 0.092	0.759 $\pm$ 0.092	0.900 $\pm$ 0.066
Colon	0.4	0.975 $\pm$ 0.054	0.991 $\pm$ 0.010	0.729 $\pm$ 0.161	0.875 $\pm$ 0.090
Lymphoma	0.2	0.957 $\pm$ 0.081	0.994 $\pm$ 0.008	0.843 $\pm$ 0.124	0.911 $\pm$ 0.066
Lymphoma	0.3	0.888 $\pm$ 0.143	0.987 $\pm$ 0.017	0.845 $\pm$ 0.121	0.942 $\pm$ 0.063
Lymphoma	0.4	0.887 $\pm$ 0.201	0.989 $\pm$ 0.024	0.846 $\pm$ 0.116	0.905 $\pm$ 0.065
DLBCL	0.2	0.98 $\pm$ 0.022	0.931 $\pm$ 0.086	0.856 $\pm$ 0.058	0.98 $\pm$ 0.032
DLBCL	0.3	0.935 $\pm$ 0.057	0.980 $\pm$ 0.025	0.857 $\pm$ 0.086	0.933 $\pm$ 0.07
DLBCL	0.4	0.995 $\pm$ 0.005	0.967 $\pm$ 0.042	0.779 $\pm$ 0.121	0.96 $\pm$ 0.064
Brain1	0.2	0.995 $\pm$ 0.005	0.951 $\pm$ 0.082	0.767 $\pm$ 0.099	0.944 $\pm$ 0.059
Brain1	0.3	0.853 $\pm$ 0.313	0.873 $\pm$ 0.240	0.844 $\pm$ 0.061	0.939 $\pm$ 0.041
Brain1	0.4	0.946 $\pm$ 0.056	0.951 $\pm$ 0.098	0.833 $\pm$ 0.124	0.889 $\pm$ 0.101
Drivface	0.2	0.964 $\pm$ 0.059	0.999 $\pm$ 0.001	0.988 $\pm$ 0.010	0.963 $\pm$ 0.018
Drivface	0.3	0.854 $\pm$ 0.320	0.999 $\pm$ 0.004	0.960 $\pm$ 0.034	0.959 $\pm$ 0.021
Drivface	0.4	0.965 $\pm$ 0.076	0.972 $\pm$ 0.071	0.929 $\pm$ 0.042	0.969 $\pm$ 0.018
Prostate	0.2	0.779 $\pm$ 0.212	0.966 $\pm$ 0.071	0.795 $\pm$ 0.106	0.895 $\pm$ 0.069
Prostate	0.3	0.946 $\pm$ 0.080	0.973 $\pm$ 0.057	0.714 $\pm$ 0.099	0.855 $\pm$ 0.069
Prostate	0.4	0.787 $\pm$ 0.228	0.919 $\pm$ 0.204	0.756 $\pm$ 0.043	0.860 $\pm$ 0.074
GLI_85	0.2	0.998 $\pm$ 0.004	0.923 $\pm$ 0.127	0.741 $\pm$ 0.115	0.900 $\pm$ 0.079
GLI_85	0.3	0.928 $\pm$ 0.157	0.953 $\pm$ 0.077	0.741 $\pm$ 0.135	0.894 $\pm$ 0.077
GLI_85	0.4	0.999 $\pm$ 0.001	0.989 $\pm$ 0.019	0.788 $\pm$ 0.142	0.835 $\pm$ 0.091
win/tie/loss	0.2	2/7/1	-/-/-	6/3/1	-/-/-
	0.3	2/8/0	-/-/-	8/2/0	-/-/-
	0.4	1/7/2	-/-/-	6/4/0	-/-/-

matrix is deleted. From this table, we can find that since we utilize the designed loss matrix in LMAO-PU, it can achieve the highest classification accuracy on almost of datasets, which shows the better classification performance than LMAO-PU-NoLM. The reason lies in the fact that in the designed loss matrix, for each U sample, k neighbors are selected and used to determine the label of the U sample, with which to tackle the issue of label uncertainty and ensure the performance of PU classifiers. Moreover, the comparison results also reveal that the sparsity obtained by LMAO-PU is comparable to LMAO-PU-NoLM in most cases. The statistics in Table 7 have verified the effectiveness of the suggested loss matrix.

4.6. Sensitivity analysis of parameters

In LMAO-PU, there are two main parameters k and gen . Specifically, the parameter k represents the number of PU samples selected as the neighbors of each U sample, with which to predict the label of U sample. The parameter gen is the number of iterations used for controlling the alternation optimization of PU model and sparse model. To perform the parameter sensitivity of k and gen , a widely used parameter sensitivity analysis method [12] is adopted, where the important parameter combinations of k and gen are tested. Fig. 6 gives

the results of parametric study, where the parameter k is changed from 5 to 20 with an interval of 5 and the other parameter gen varies from 10 to 50 with an interval of 10. The best and worst classification results are also marked in red, which are also shown in this figure. From this figure, we can find that the two parameters are not very sensitive to the performance of LMAO-PU, since on most of the datasets, the best and worst results only have a little difference, which means that the two parameters are easily tuned in practical applications. From the results shown in Fig. 6, we can find that when the parameters (k , gen) are set to (10, 20), the performance of the obtained PU classifier is better on most of the datasets. Thus, in our experiments, the parameters k and gen are set to 10 and 20, respectively.

5. Conclusion and future work

To tackle PU learning under high-dimensional PU data situation, this paper has proposed an alternating optimization method, named LMAO-PU. In LMAO-PU, two models, i.e., PU model and sparse model, were built and optimized alternately to obtain the optimal sparse PU classifier. To implement the suggested method, a loss matrix was suggested to evaluate the performance of PU classifiers on U samples. With the proposed loss matrix, the two models were formulated as

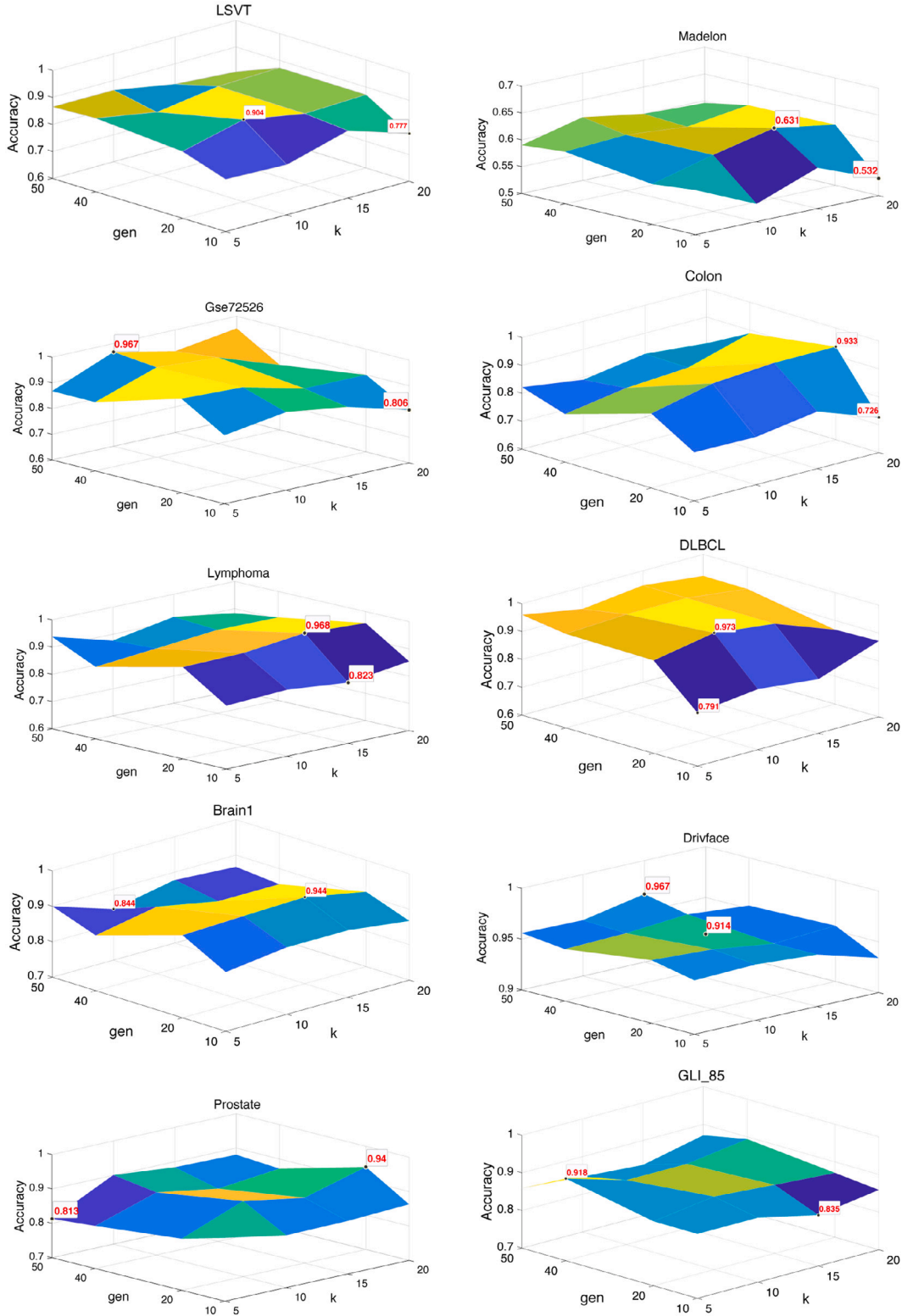


Fig. 6. Parameter sensitivity analysis of parameters (k , gen) on the 10 datasets.

two bi-objective optimization problems and an MOEA was employed to solve them. Compared with six state-of-the-art PU algorithms, the experimental results in terms of sparsity, accuracy and AUC on 10 real-world high-dimensional datasets have demonstrated the effectiveness of the proposed LMAO-PU. The conclusions further indicate LMAO-PU

can achieve sparse PU classifiers with high accuracy and sparsity, which can be better used for real PU applications.

The proposed algorithm has shown promising performance in our experiments. However, since it adopts the population-based MOEA framework, which is similar to the other MOEAs in real applications [45], the computing cost of proposed algorithm is large. Especially

for large scale PU data, the computation efficiency is obviously decreased, which will be tackled in the future work. Additionally, in our work, the issue of noisy P samples is not considered. It is desirable to design an efficient PU algorithm for PU learning from noisy PU data, which will also be one of the future work.

CRedit authorship contribution statement

Jianfeng Qiu: Methodology, Software, Original draft preparation. **Xiaoqiang Cai:** Data curation, Experiments. **Lei Zhang:** Visualization, Investigation, Reviewing. **Fan Cheng:** Supervision, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (Grant No. 62076001, 61976001, 61876184, U1804262), Humanities and Social Sciences Project of Chinese Ministry of Education (Grant No. 18YJC870004), the Natural Science Foundation of Anhui Province, China (Grant No. 1708085MF166, 1908085MF218 and 2008085QF308), the Anhui Provincial Key Research and Development Plan, China (Grand No. 202004j07020005), the University Synergy Innovation Program of Anhui Province, China (GXXT-2020-050) and the Key Program of Natural Science Project of Educational Commission of Anhui Province, China (Grant no. KJ2021A0634, KJ2021A0048).

References

- [1] Luis M. de Campos, Juan M. Fernandez Luna, Juan F. Huete, Luis Redondo Exposito, Positive unlabeled learning for building recommender systems in a parliamentary setting, *Inf. Sci.* 433 (2018) 221–232.
- [2] Mariana Caravanti de Souza, Bruno Magalhaes Nogueira, Rafael Geraldini Rossi, Ricardo Marcondes Marcacini, Bruce Neves dos Santos, Solange Oliveira Rezende, A network-based positive and unlabeled learning approach for fake news detection, *Mach. Learn.* 1–44.
- [3] Yang Zhang, Yang Qiu, Yuxin Cui, Shichao Liu, Wen Zhang, Predicting drug-drug interactions using multi-modal deep auto-encoders based network embedding and positive-unlabeled learning, *Methods* 179 (2020) 37–46.
- [4] Xin Feng, Jialiang Li, Han Li, Hang Chen, Fei Li, Quewang Liu, ZhuHong You, Fengfeng Zhou, Age is important for the early-stage detection of breast cancer on both transcriptomic and methylomic biomarkers, *Front. Genet.* 10 (212) (2019) 1–13.
- [5] Jessa Bekker, Jesse Davis, Learning from positive and unlabeled data: a survey, *Mach. Learn.* 109 (4) (2020) 719–760.
- [6] Hwanjo Yu, Jiawei Han, Kevin Chen chuan, PEBL: positive example based learning for web page classification using SVM, in: *International Conference on Knowledge Discovery and Data Mining*, 2002, pp. 239–248.
- [7] Xiaoli Li, Bing Liu, Learning to classify texts using positive and unlabeled data, in: *International Joint Conference on Artificial Intelligence*, Vol. 3, 2003, pp. 587–592.
- [8] Bing Liu, Yang Dai, Xiaoli Li, Wee Sun Lee, S Yu Philip, Building text classifiers using positive and unlabeled examples, in: *International Conference on Data Mining*, Vol. 3, 2003, pp. 179–188.
- [9] Chuan Luo, Pu Zhao, Chen Chen, Bo Qiao, Chao Du, Hongyu Zhang, Wei Wu, Shaowei Cai, Bing He, Saravanakumar Rajmohan, Qingwei Lin, PULNS: positive-unlabeled learning with effective negative sample selector, in: *International Conference on Artificial Intelligence*, 2021, pp. 8784–8792.
- [10] Charles Elkan, Keith Noto, Learning classifiers from only positive and unlabeled data, in: *International Conference on Knowledge Discovery and Data Mining*, 2008, pp. 213–220.
- [11] Hong Shi, Shaojun Pan, Jian Yang, Chen Gong, Positive and unlabeled learning via loss decomposition and centroid estimation, in: *International Joint Conference on Artificial Intelligence*, 2018, pp. 2689–2695.
- [12] Chen Gong, Hong Shi, Tongliang Liu, Chuang Zhang, Jian Yang, Dacheng Tao, Loss decomposition and centroid estimation for positive and unlabeled learning, *IEEE Trans. Pattern Anal. Mach. Intell.* 43 (2019) 918–932.
- [13] Curtis G. Northcutt, Lu Jiang, Isaac L. Chuang, Confident learning: Estimating uncertainty in dataset labels, 2019, *ArXiv:1911.00068v2*.
- [14] Marthinus C. Du Plessis, Gang Niu, Masashi Sugiyama, Convex formulation for learning from positive and unlabeled data, in: *International Conference on Machine Learning*, 2015, pp. 1386–1394.
- [15] Ryuichi Kiryo, Gang Niu, Marthinus C. Du Plessis, Masashi Sugiyama, Positive-unlabeled learning with non-negative risk estimator, in: *Advances in Neural Information Processing Systems*, 2017, pp. 1675–1685.
- [16] Yu Guan Hsieh, Gang Niu, Masashi Sugiyama, Classification from positive, unlabeled and biased negative data, in: *International Conference on Machine Learning*, 2019, pp. 2820–2829.
- [17] Jianfeng Qiu, Xiaoqiang Cai, Xingyi Zhang, Fan Cheng, Shenzhi Yuan, Guanglong Fu, An evolutionary multi-objective approach to learn from positive and unlabeled data, *Appl. Soft Comput.* 101 (2021) 106986.
- [18] Yi Zheng, Hui Peng, Xiaocai Zhang, Zhixun Zhao, Xiaoying Gao, Jinyan Li, DDI-PULearn: a positive-unlabeled learning method for large-scale prediction of drug-drug interactions, *BMC Bioinformatics* 20 (19) (2019) 1–12.
- [19] Akram Vasighizaker, Saeed Jalili, C-PUGP: A cluster-based positive unlabeled learning method for disease gene prediction and prioritization, *Comput. Biol. Chem.* 76 (2018) 23–31.
- [20] Jiaqi Zhang, Zhenzhen Wang, Jingjing Meng, Yap-Peng Tan, Junsong Yuan, Boosting positive and unlabeled learning for anomaly detection with multi-features, *IEEE Trans. Multimed.* 21 (5) (2018) 1332–1344.
- [21] Peng Yang, Xiao Li Li, Jian Ping Mei, Chee Keong Kwoh, See Kiong Ng, Positive-unlabeled learning for disease gene identification, *Bioinformatics* 28 (20) (2012) 2640–2647.
- [22] William S. Noble, What is a support vector machine? *Nature Biotechnol.* 24 (12) (2006) 1565–1567.
- [23] Julio César Carnevali, Rafael Geraldini Rossi, Evangelos Milios, Alneu de Andrade Lopes, A graph-based approach for positive and unlabeled learning, *Inform. Sci.* 580 (2021) 655–672.
- [24] Chen Gong, Tongliang Liu, Jian Yang, Dacheng Tao, Large-margin label-calibrated support vector machines for positive and unlabeled learning, *IEEE Trans. Neural Netw. Learn. Syst.* 30 (11) (2019) 3471–3483.
- [25] Chen Gong, Qizhou Wang, Tongliang Liu, Bo Han, Jane J You, Jian Yang, Dacheng Tao, Instance-dependent positive and unlabeled learning with labeling bias estimation, *IEEE Trans. Pattern Anal. Mach. Intell.* (2021).
- [26] Bo Liu, Changdong Liu, Yanshan Xiao, Laiwang Liu, Weinan Li, Xiaodong Chen, AdaBoost-based transfer learning method for positive and unlabeled learning problem, *Knowl.-Based Syst.* (2022) 108162.
- [27] Wee Sun Lee, Bing Liu, Learning with positive and unlabeled examples using weighted logistic regression, in: *International Conference on Machine Learning*.
- [28] Curtis Northcutt, Lu Jiang, Isaac Chuang, Confident learning: Estimating uncertainty in dataset labels, *J. Artificial Intelligence Res.* 70 (2021) 1373–1411.
- [29] P. Yang, J.T. Ormerod, W. Liu, C. Ma, A.Y. Zomaya, J.Y.H. Yang, AdaSampling for positive-unlabeled and label noise learning with bioinformatics applications, *IEEE Trans. Cybern.* 49 (5) (2019) 1932–1943.
- [30] Marthinus C. Du Plessis, Gang Niu, Masashi Sugiyama, Analysis of learning from positive and unlabeled data, in: *Advances in Neural Information Processing Systems*, 2014, pp. 703–711.
- [31] Xiuhua Chen, Chen Gong, Jian Yang, Cost-sensitive positive and unlabeled learning, *Inform. Sci.* 558 (2021) 229–245.
- [32] Chuang Zhang, Dexin Ren, Tongliang Liu, Jian Yang, Chen Gong, Positive and unlabeled learning with label disambiguation, in: *International Joint Conference on Artificial Intelligence*, 2019, pp. 4250–4256.
- [33] Tomoya Sakai, Marthinus Christoffel Du Plessis, Gang Niu, Masashi Sugiyama, Semi-supervised classification based on classification from positive and unlabeled data, in: *International Conference on Machine Learning*, Vol. 70, 2017, pp. 2998–3006.
- [34] Jessa Bekker, Jesse Davis, Estimating the class prior in positive and unlabeled data through decision tree induction, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32, 2018.
- [35] Shizhen Chang, Bo Du, Liangpei Zhang, Positive unlabeled learning with class-prior approximation, in: *Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence*, 2021, pp. 2014–2021.
- [36] B. Xue, M. Zhang, W.N. Browne, Particle swarm optimization for feature selection in classification: a multi-objective approach, *IEEE Trans. Cybern.* 43 (6) (2013) 1656–1671.
- [37] Leif E. Peterson, K-nearest neighbor, *Scholarpedia* 4 (2) (2009) 1883.
- [38] Fan Cheng, Guanglong Fu, Xingyi Zhang, Jianfeng Qiu, Multi-objective evolutionary algorithm for optimizing the partial area under the ROC curve, *Knowl.-Based Syst.* 170 (2019) 61–69.
- [39] AnDa Li, Bing Xue, Mengjie Zhang, Multi-objective feature selection using hybridization of a genetic algorithm and direct multi-search for key quality characteristic selection, *Inf. Sci.* 523 (2020) 245–265.
- [40] K. Deb, A. Pratap, S. Agarwal, T.A.M.T. Meyarivan, A fast and elitist multi-objective genetic algorithm: NSGA-II, *IEEE Trans. Evol. Comput.* 6 (2) (2002) 182–197.
- [41] Kalyanmoy Deb, Multi-objective optimisation using evolutionary algorithms: an introduction, in: *Multi-Objective Evolutionary Optimisation for Product Design and Manufacturing*, 2011, pp. 3–34.

- [42] Kalyanmoy Deb, Mayank Goyal, A combined genetic adaptive search (GeneAS) for engineering design, *Comput. Sci. Inf.* 26 (1996) 30–45.
- [43] Chuang Zhang, Chen Gong, Tengfei Liu, Xun Lu, Weiqiang Wang, Jian Yang, On-line positive and unlabeled learning, in: *Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence*, 2021, pp. 2248–2254.
- [44] Ke Ren, Haichuan Yang, Yu Zhao, Wu Chen, Mingshan Xue, Hongyu Miao, Shuai Huang, Ji Liu, A robust AUC maximization framework with simultaneous outlier detection and feature selection for positive-unlabeled classification, *IEEE Trans. Neural Netw. Learn. Syst.* 30 (10) (2019) 3072–3083.
- [45] Bo-Yang Qu, Y.S. Zhu, Y.C. Jiao, M.Y. Wu, Ponnuthurai N Suganthan, Jing J Liang, A survey on multi-objective evolutionary algorithms for the solution of the environmental/economic dispatch problems, *Swarm Evol. Comput.* 38 (2018) 1–11.